EXP NO:1)A Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.

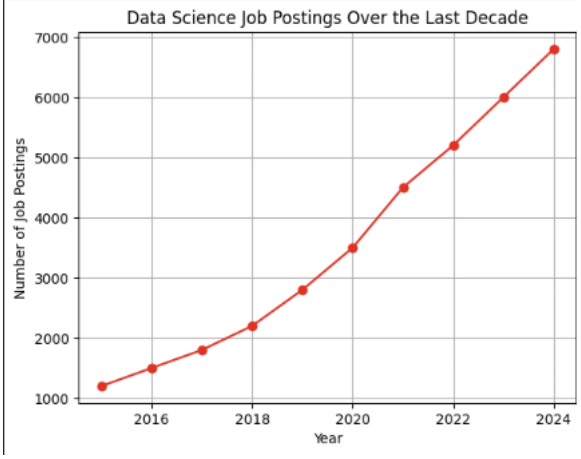
AIM:

To Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Year": [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
    "Job_Postings": [1200, 1500, 1800, 2200, 2800, 3500, 4500, 5200, 6000, 6800]
}
df = pd.DataFrame(data)
print(df)
plt.plot(df["Year"], df["Job_Postings"], marker='o',color="red")
plt.title("Data Science Job Postings Over the Last Decade")
plt.xlabel("Year")
plt.ylabel("Number of Job Postings")
plt.grid(True)
plt.show()
```

```
   Year  Job_Postings
0  2015          1200
1  2016          1500
2  2017          1800
3  2018          2200
4  2019          2800
5  2020          3500
6  2021          4500
7  2022          5200
8  2023          6000
9  2024          6800
```



RESULT:

Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /sea
born for visualization.

EXP NO:1)B Analyze and visualize the distribution of various data science roles (Data

Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

AIM:

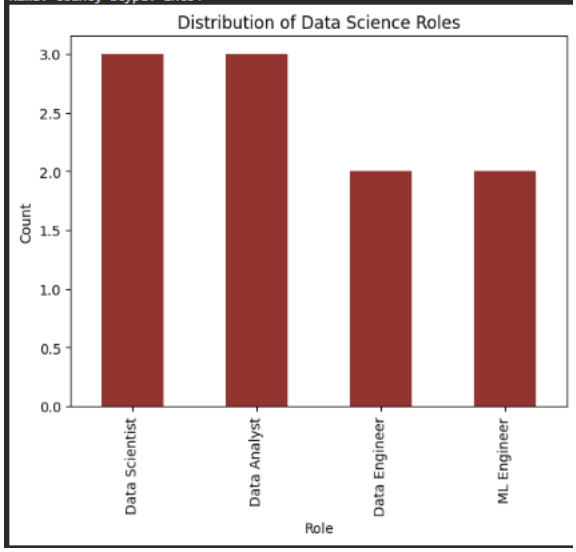
To analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Role": ["Data Scientist", "Data Analyst", "Data Engineer",
             "ML Engineer", "Data Analyst", "Data Scientist",
             "Data Engineer", "Data Scientist", "ML Engineer",
             "Data Analyst"]
}
df = pd.DataFrame(data)
role_count = df["Role"].value_counts()
print(role_count)
role_count.plot(kind="bar", color="brown")
plt.title("Distribution of Data Science Roles")
plt.xlabel("Role")
plt.ylabel("Count")
plt.show()
```

OUTPUT:

```
Role
Data Scientist    3
Data Analyst      3
Data Engineer     2
ML Engineer       2
Name: count, dtype: int64
```



RESULT:

Thus the program Analyze and visualize the distribution of various data science roles from a dataset.

EXP NO:1)C Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

AIM:

To Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

PROGRAM:

```
import pandas as pd
structured_data = pd.DataFrame({
    "Name": ["John", "Mary", "Sam"],
    "Age": [20, 25, 22]
})
semi_structured_data = {
    "Name": "John",
    "Age": 20,
    "City": "Chennai"
}
unstructured_data = [
    "Hello, this is a text file",
    "Photo.jpg",
    "Video.mp4"
]
print("Structured Data:")
print(structured_data)
print("\nSemi-Structured Data:")
print(semi_structured_data)
print("\nUnstructured Data:")
print(unstructured_data)
```

OUTPUT:

```
Structured Data:
   Name  Age
0  John   20
1  Mary   25
2   Sam   22
```

Semi-Structured Data:

```
{'Name': 'John', 'Age': 20, 'City': 'Chennai'}
```

Unstructured Data:

```
['Hello, this is a text file', 'Photo.jpg', 'Video.mp4']
```

RESULT:

Thus the program to Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

EXP NO:1)D Conduct an experiment to encrypt and decrypt given sensitive data Use cryptography library fernet.

AIM:

To Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.

PROGRAM:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
cipher = Fernet(key)
data = "My password is 12345"
encrypted_data = cipher.encrypt(data.encode())
decrypted_data = cipher.decrypt(encrypted_data).decode()
print("Original Data:")
print(data)
print("\nEncrypted Data:")
print(encrypted_data)
print("\nDecrypted Data:")
print(decrypted_data)
```

OUTPUT:

```
Original Data:
My password is 12345

Encrypted Data:
b'gAAAAABqadsmPyzIuKKli9XeWhRJgqQIHXtcvkbCOgnQnpGRoqPh4k28pqIJyGW
IFfREL4D0OV0dLFV9sYMQfGn2HZvEe3RFIwgzImlrmG2P7y-LM3TbRx0='

Decrypted Data:
My password is 12345
```

RESULT:

Thus the program to Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.

EXP NO:2 Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

AIM:

To upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
print("File Loaded Successfully:", file_name)
print("\nFirst 5 rows of the dataset:")
print(df.head())
print("\nMissing values:")
print(df.isnull().sum())
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(
    subset=['Product', 'Quantity', 'Region'],
    inplace=True
)
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df.dropna(subset=['Date'], inplace=True)
print("\nMissing values after preprocessing:")
print(df.isnull().sum())
print("\nSummary Statistics:")
print(df.describe())
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()
print("\nProduct-wise Sales and Quantity:")
print(product_summary)
plt.figure(figsize=(10, 6))
plt.bar(
    product_summary['Product'],
    product_summary['Sales'],
    color='steelblue'
)
plt.xlabel('Product')
plt.ylabel('Total Sales')
```

```
plt.title('Total Sales by Product')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
sales_over_time = df.groupby('Date').agg({
    'Sales': 'sum'
}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(
    sales_over_time['Date'],
    sales_over_time['Sales'],
    marker='o',
    color='green'
)
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)
print("\nSales by Region and Product:")
print(pivot_table)
numeric_df = df.select_dtypes(include=np.number)
correlation_matrix = numeric_df.corr()
print("\nCorrelation Matrix:")
print(correlation_matrix)
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
print("\nFinal Preprocessed Dataset:")
print(df.head())
print("\nFinal Dataset Shape:", df.shape)
print("\nData preprocessing and visualization completed successfully!")
```

OUTPUT:

First 5 rows of the dataset:

	Date	Product	Quantity	Sales	Region
0	2026-01-01	Laptop	2	120000.0	North
1	2026-01-02	Phone	5	75000.0	South
2	2026-01-03	Headphones	8	16000.0	East
3	2026-01-04	Tablet	3	45000.0	West
4	2026-01-05	Laptop	1	60000.0	East

Missing values:

Date	0
Product	1
Quantity	0
Sales	1
Region	1

dtype: int64

Missing values after preprocessing:

Date	0
Product	0
Quantity	0
Sales	0
Region	0

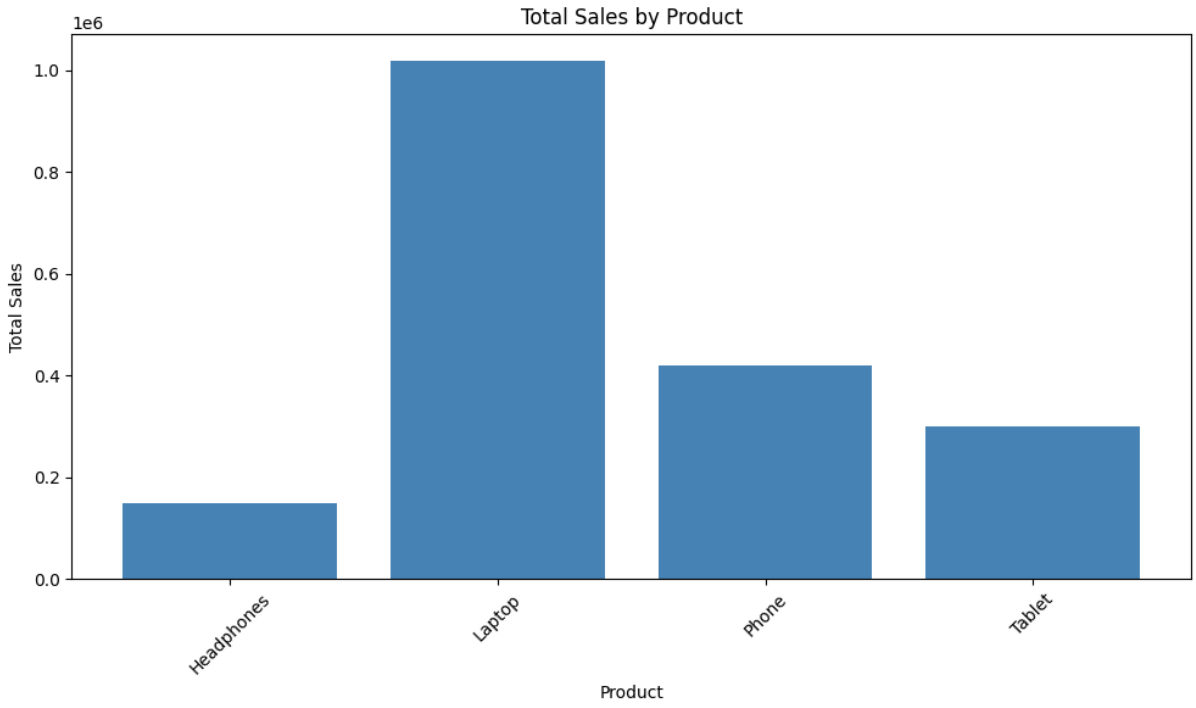
dtype: int64

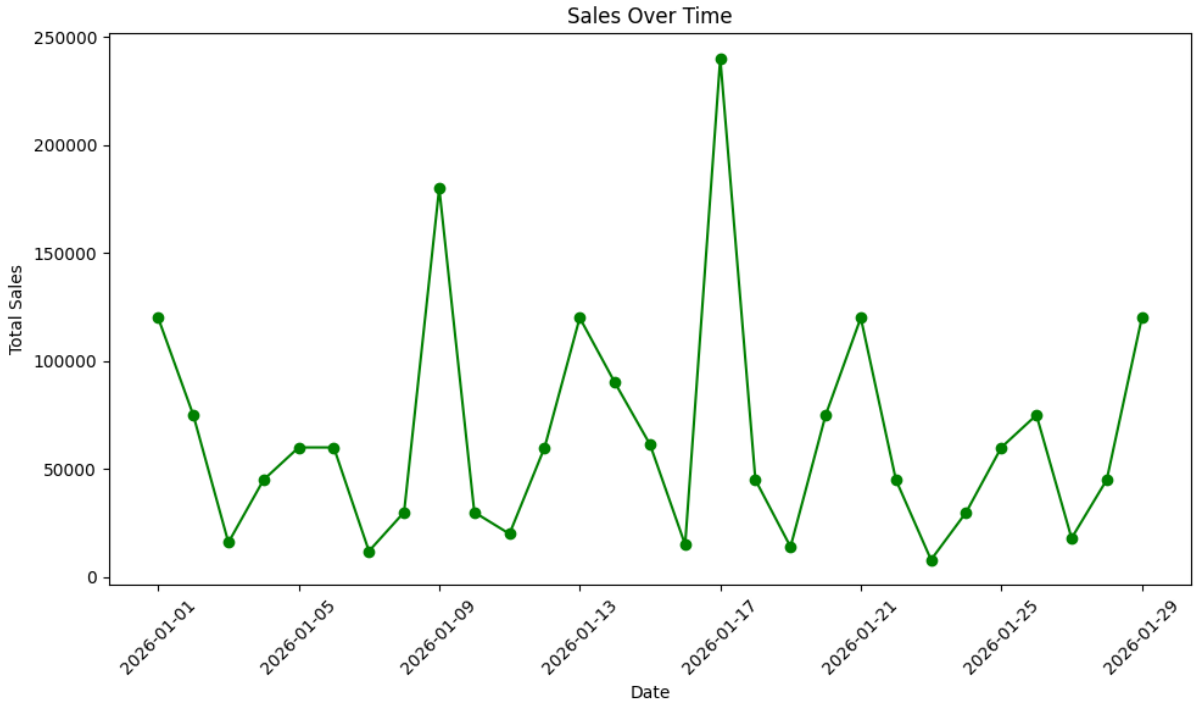
Summary Statistics:

	Date	Quantity	Sales
count	29	29.000000	29.000000
mean	2026-01-15 00:00:00	3.931034	65151.279199
min	2026-01-01 00:00:00	1.000000	8000.000000
25%	2026-01-08 00:00:00	2.000000	30000.000000
50%	2026-01-15 00:00:00	3.000000	60000.000000
75%	2026-01-22 00:00:00	5.000000	75000.000000
max	2026-01-29 00:00:00	10.000000	240000.000000
std	NaN	2.374433	53471.083438

Product-wise Sales and Quantity:

	Product	Sales	Quantity
0	Headphones	1.493871e+05	49
1	Laptop	1.020000e+06	17
2	Phone	4.200000e+05	28
3	Tablet	3.000000e+05	20





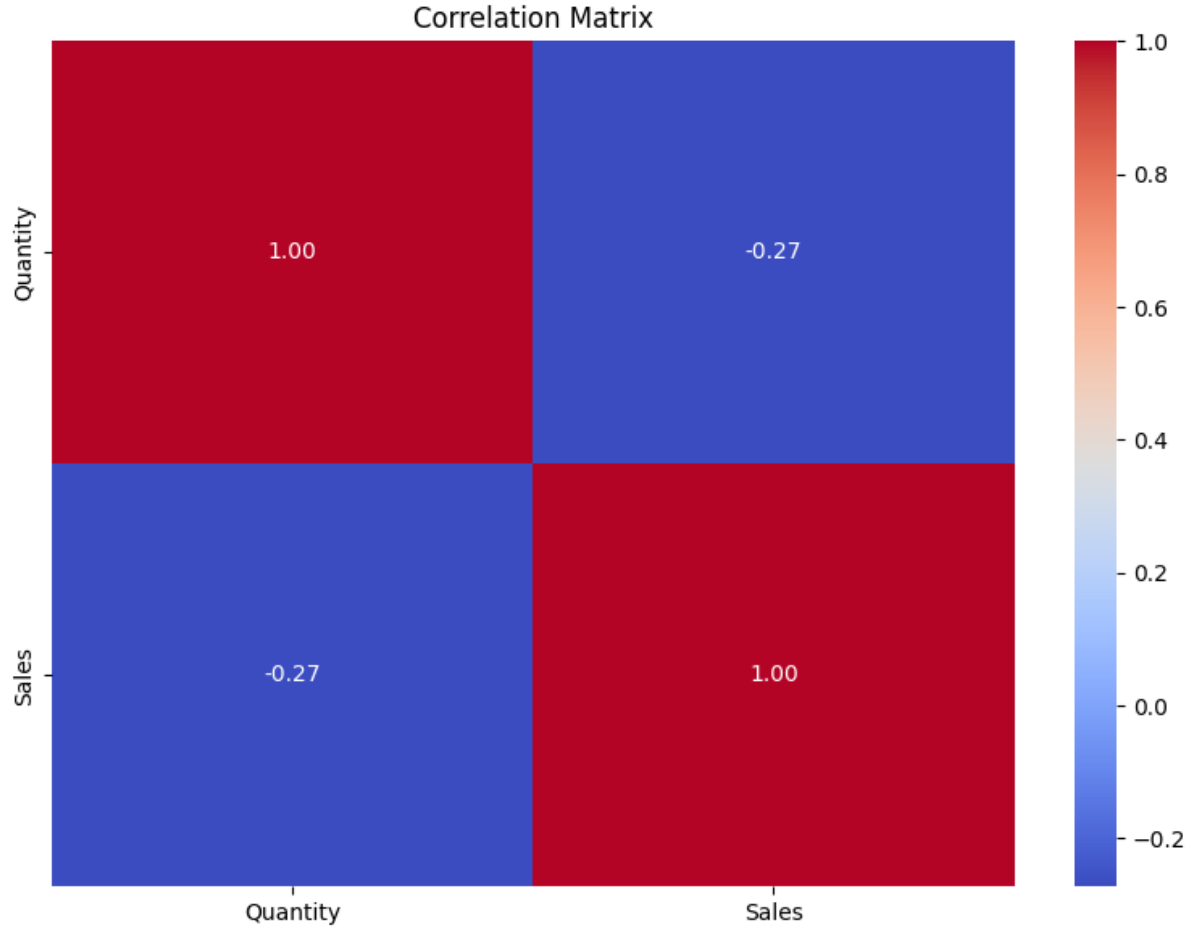
Sales by Region and Product:

Region	Product	Headphones	Laptop	Phone	Tablet
East		30000.000000	120000.0	75000.0	15000.0
North		61387.096774	480000.0	135000.0	30000.0
South		20000.000000	120000.0	120000.0	105000.0
West		38000.000000	300000.0	90000.0	150000.0

Correlation Matrix:

	Quantity	Sales

Quantity 1.000000 -0.272382
Sales   -0.272382 1.000000



Final Preprocessed Dataset:

	Date	Product	Quantity	Sales	Region
0	2026-01-01	Laptop	2	120000.0	North
1	2026-01-02	Phone	5	75000.0	South
2	2026-01-03	Headphones	8	16000.0	East
3	2026-01-04	Tablet	3	45000.0	West
4	2026-01-05	Laptop	1	60000.0	East

Final Dataset Shape: (29, 5)

Data preprocessing and visualization completed successfully!

RESULT:

Thus the program to Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

EXP NO:3)A Conduct an experiment to show data visualization using line plot

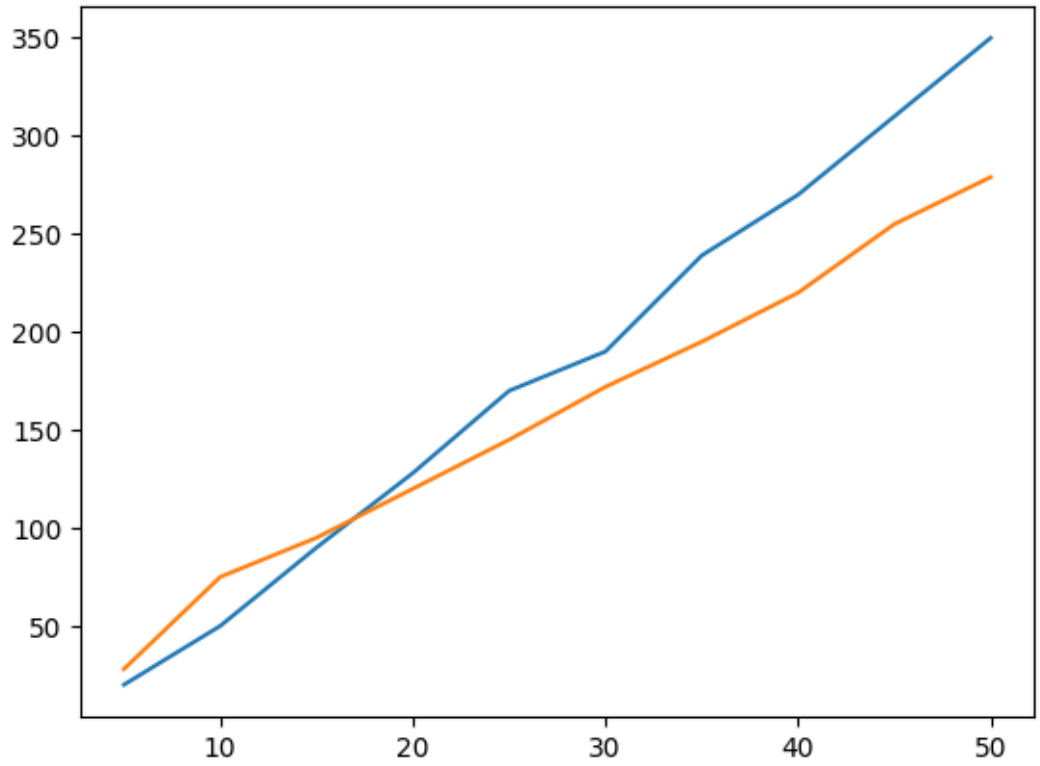
AIM:

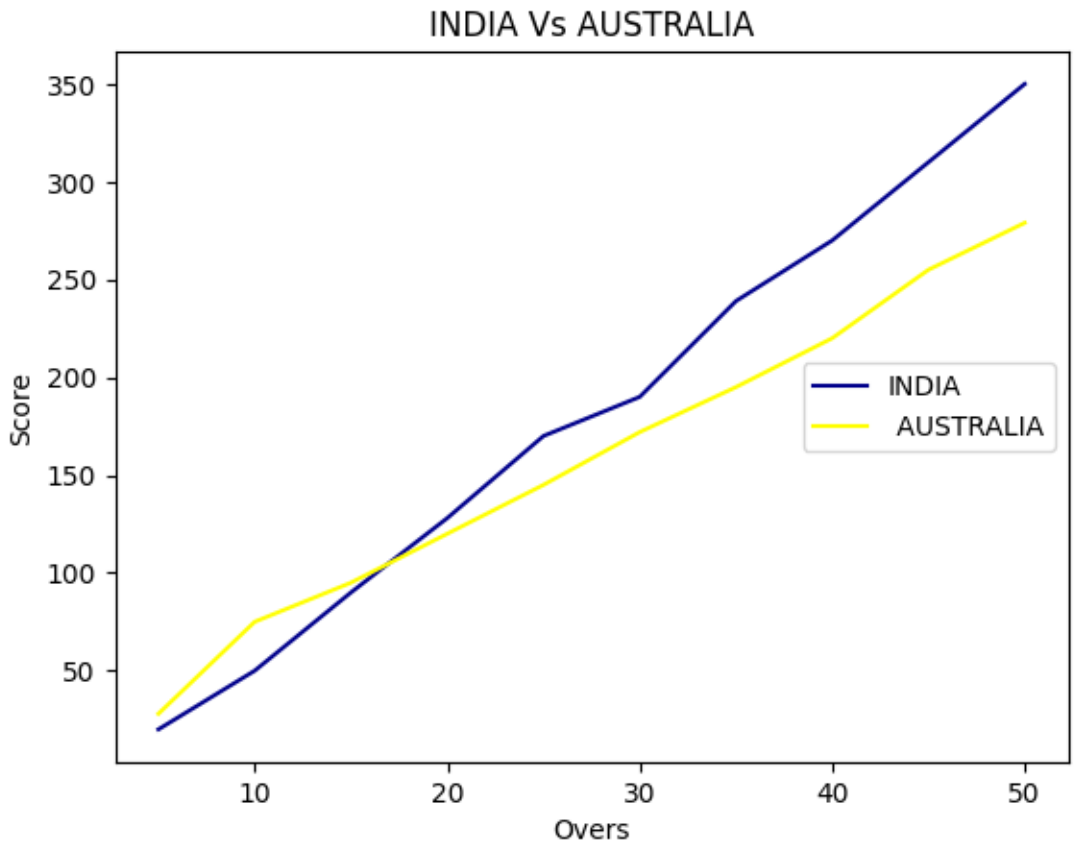
To conduct an experiment to show data visualization using line plot

PROGRAM:

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[20,50,90,128,170,190,239,270,310,350]
Srilankan_Score=[28,75,95,120,145,172,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs AUSTRALIA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="darkblue",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="yellow",label=" AUSTRALIA")
cricket.legend(loc="center right")
```

OUTPUT:





RESULT:

Thus the program to Conduct an experiment to show data visualization using line plot is done

EXP NO:3)B Conduct an experiment to show data visualization using bar chart

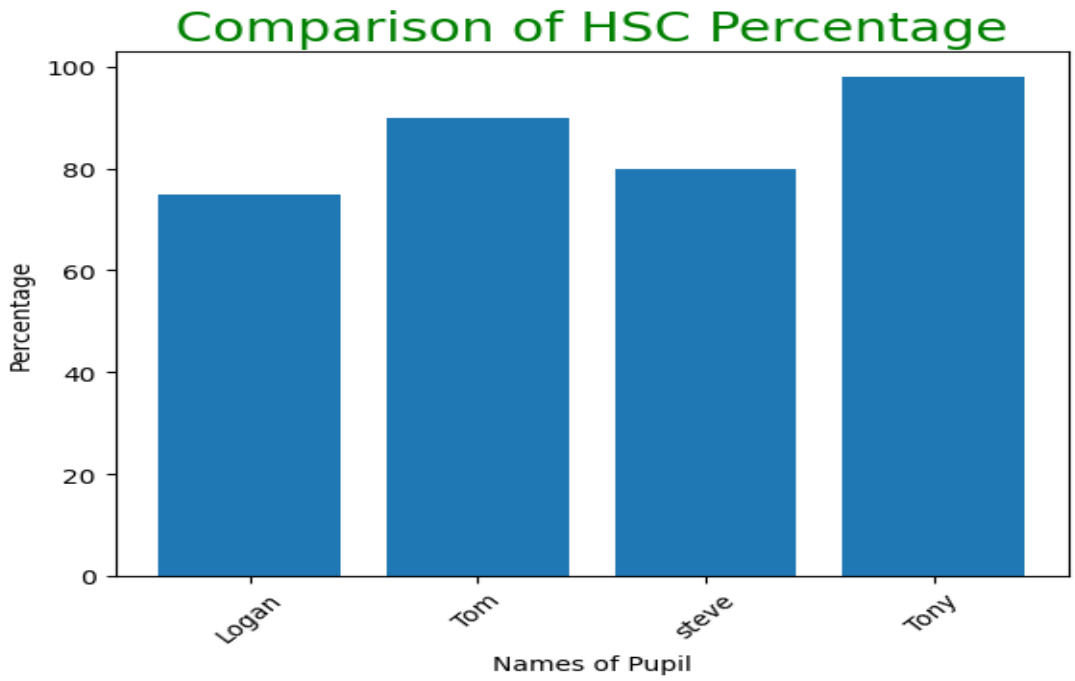
AIM:

To conduct an experiment to show data visualization using bar chart

PROGRAM:

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['Logan', 'Tom', 'steve', 'Tony']
xaxis = np.arange(len(Names))
Percentage_hsc = [75,90,80,98]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

OUTPUT:



RESULT:

Thus the program to conduct an experiment to show data visualization using bar chart is done

EXP NO:3)C Conduct an experiment to show data visualization using pie chart

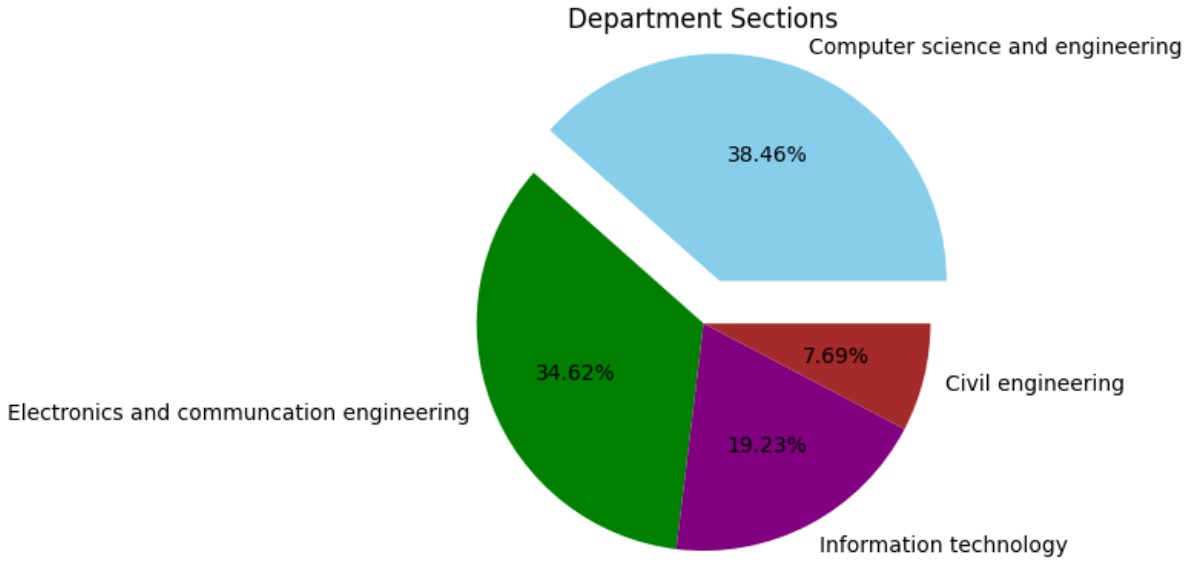
AIM:

To conduct an experiment to show data visualization using pie chart

PROGRAM:

```
import matplotlib.pyplot as plt
labels = ['Computer science and engineering', 'Electronics and communcation
engineering', 'Information technology', 'Civil engineering']
section = [10,9,5,2]
colors = ['skyblue', 'green', 'purple', 'brown']
explode = (0.2, 0, 0, 0)
plt.pie(section, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
plt.title('Department Sections')
plt.show()
```

OUTPUT:



RESULT:

Thus the program to conduct an experiment to show data visualization using pie chart is done.

EXP NO:4 To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

AIM:

To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

PROGRAM:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('pun+kt')
nltk.download('punkt_tab')
sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
    wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT:

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever', 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposition', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('existence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world', 1), ('with', 2)
```

RESULT:

Thus the program to Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Exp No:5 Data Collection and Initial Exploration

AIM:

To collect a sales dataset in CSV format and explore the data using Python. To identify missing values and visualize sales data using graphs.

PROGRAM:

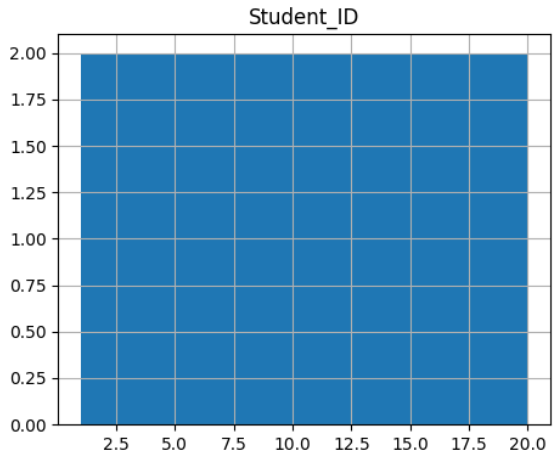
```
from google.colab import files
uploaded = files.upload()
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("student_data.csv")
print(df.head())
print(df.info())
print(df.describe())
df.hist(bins=10, figsize=(12, 15))
plt.show()
sns.pairplot(df)
plt.show()
```

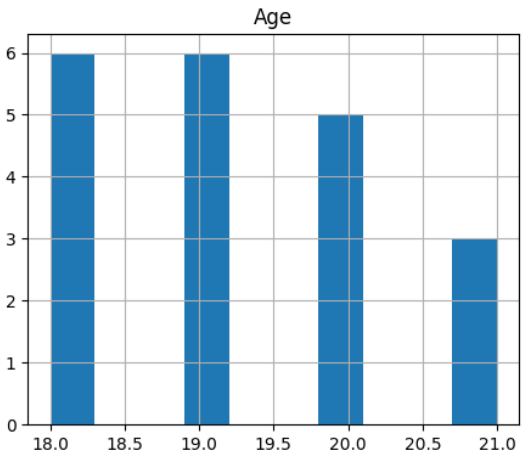
OUTPUT:

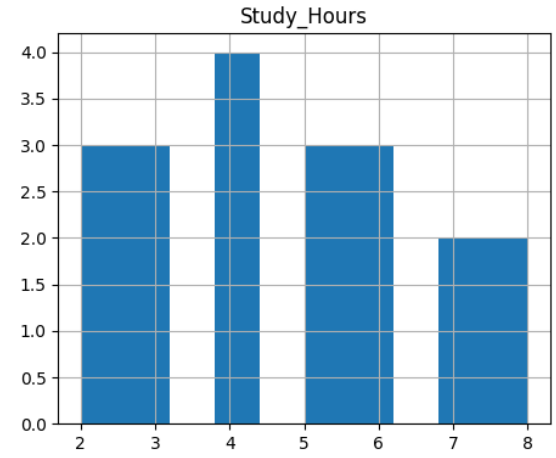
```
  Student_ID  Age  Study_Hours  Attendance  Marks  Assignments_Completed
0          1   18            2          75     55                     6
1          2   19            4          82     68                     8
2          3   18            3          78     60                     7
3          4   20            6          90     85                    10
4          5   19            5          85     76                     9
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Student_ID           20 non-null     int64
 1   Age                  20 non-null     int64
 2   Study_Hours          20 non-null     int64
 3   Attendance           20 non-null     int64
 4   Marks                20 non-null     int64
 5   Assignments_Completed  20 non-null     int64
dtypes: int64(6)
memory usage: 1.1 KB
None
       Student_ID        Age  Study_Hours  Attendance       Marks  \
count    20.00000  20.000000     20.00000      20.000  20.000000
mean     10.50000  19.250000      4.70000      84.750  74.300000
std       5.91608   1.069924      1.92217       8.309  14.733063
min       1.00000  18.000000      2.00000      70.000  50.000000
25%       5.75000  18.000000      3.00000      78.750  61.500000
```

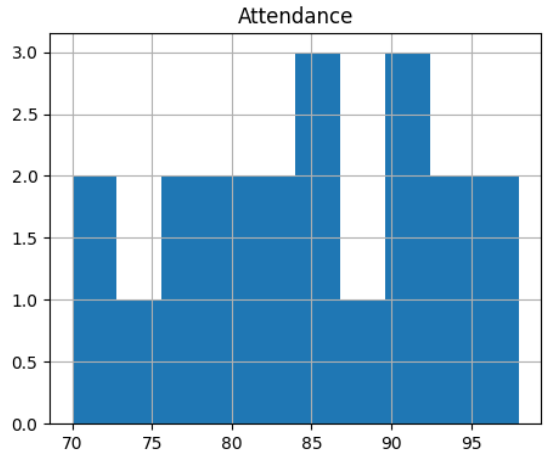
50%	10.50000	19.000000	4.50000	84.500	74.500000
75%	15.25000	20.000000	6.00000	91.250	87.250000
max	20.00000	21.000000	8.00000	98.000	96.000000

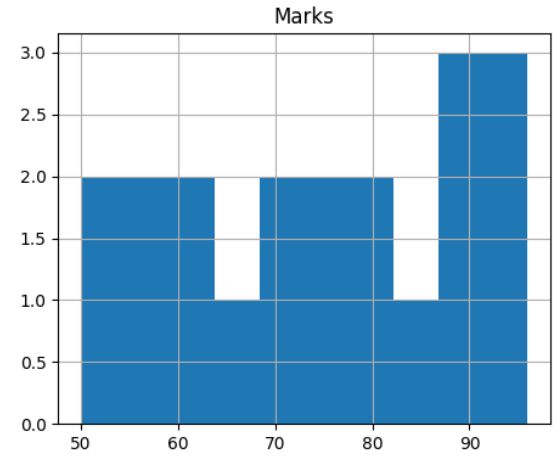
	Assignments_Completed
count	20.000000
mean	8.250000
std	1.743409
min	5.000000
25%	7.000000
50%	8.500000
75%	10.000000
max	10.000000

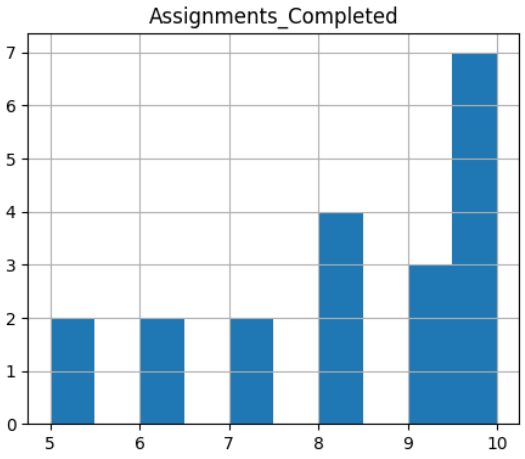

Student_ID
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

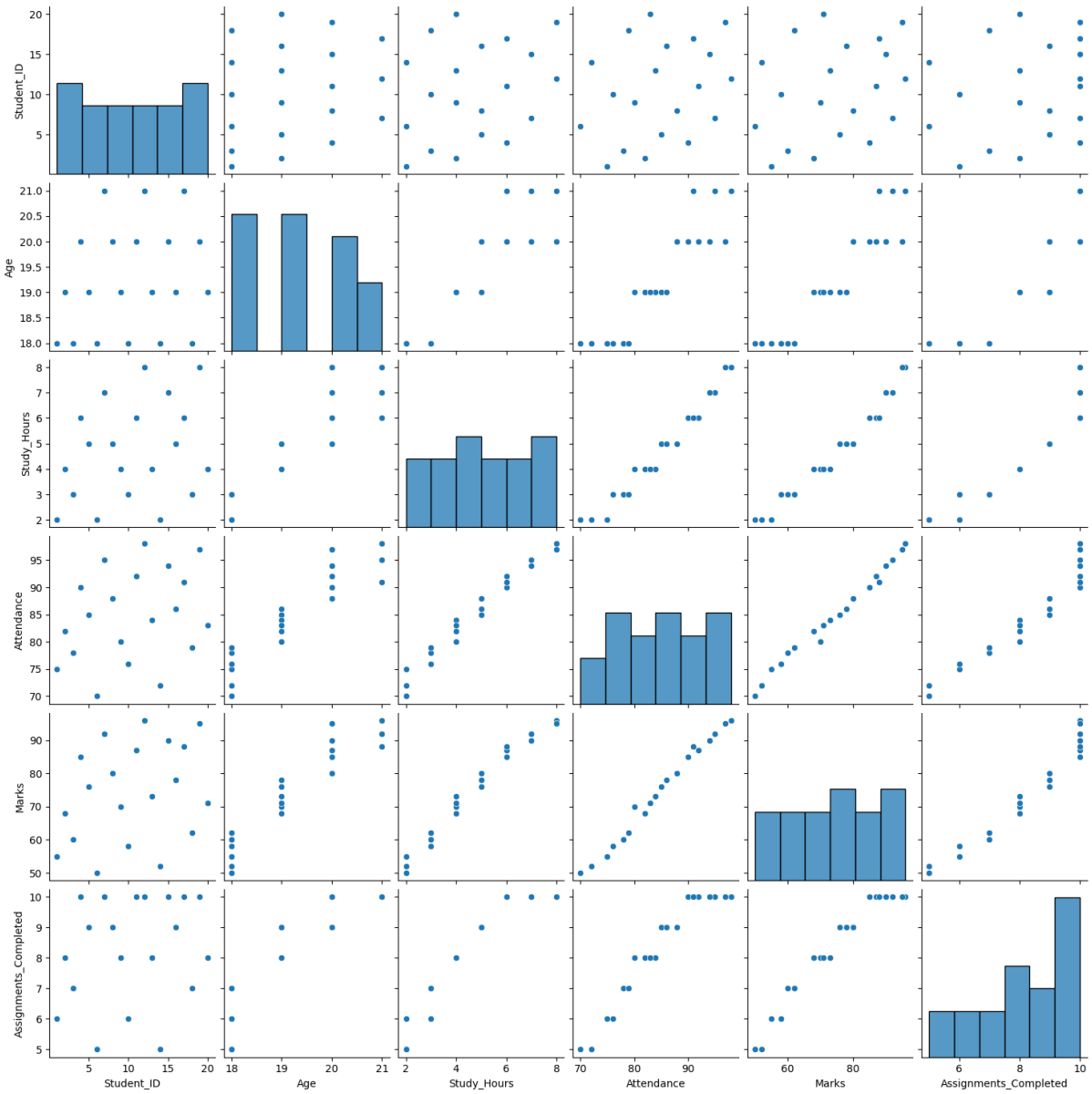

Age
6
5
4
3
2
1
0
18.0
18.5
19.0
19.5
20.0
20.5
21.0


Study_Hours
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
2
3
4
5
6
7
8


Attendance
3.0
2.5
2.0
1.5
1.0
0.5
0.0
70
75
80
85
90
95


Marks
3.0
2.5
2.0
1.5
1.0
0.5
0.0
50
60
70
80
90


Assignments_Completed
7
6
5
4
3
2
1
0
5
6
7
8
9
10



RESULT:

The student dataset was successfully loaded and explored using Pandas. Summary statistics and visualizations were generated to understand the distribution and relationships among the student data.

Experiment 6: Time Series Analysis of Website Traffic

Aim :

To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

Program Code :

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from google.colab import files

uploaded = files.upload()
# Load dataset
data = pd.read_csv(
    'website_traffic.csv',
    parse_dates=['Date'],
    index_col='Date'
)

# Time Series Decomposition
decomposition = seasonal_decompose(
    data['Page_Views'],
    model='additive',
    period=7
)

decomposition.plot()
plt.show()

# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
```

```
data[['Page_Views', 'Page_Views_MA']].plot(
    figsize=(10, 5)
)

plt.title('Page Views with Moving Average')
plt.xlabel('Date')
plt.ylabel('Page Views')
plt.show()

# Anomaly Detection using Z-score
data['Page_Views_Z'] = (
    (data['Page_Views'] - data['Page_Views'].mean())
    / data['Page_Views'].std()
)

# Find anomalies
anomalies = data[data['Page_Views_Z'].abs() > 3]

# Plot anomalies
plt.figure(figsize=(10, 5))

plt.plot(
    data.index,
    data['Page_Views'],
    label='Page Views'
)

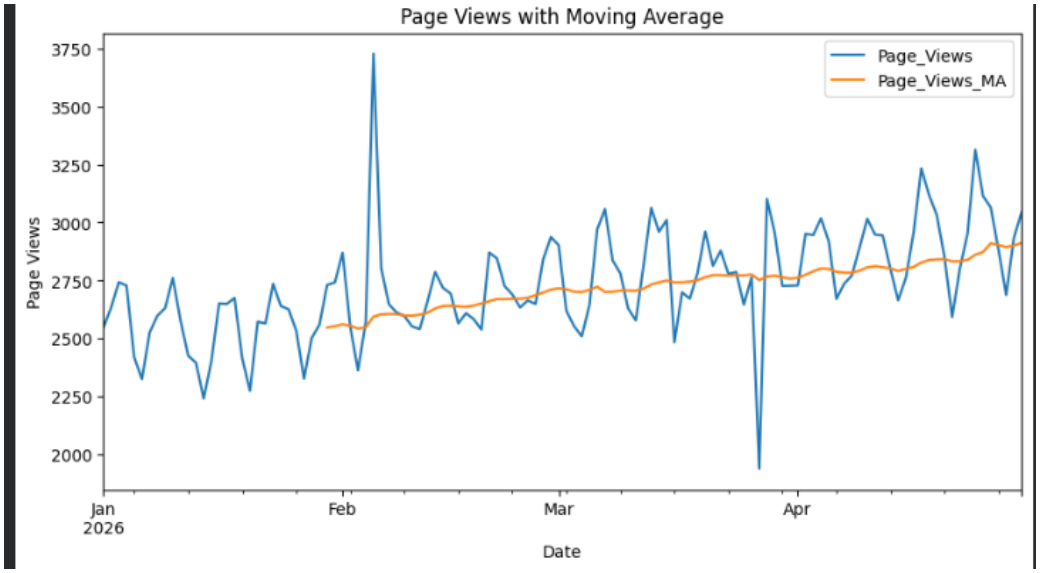
plt.scatter(
    anomalies.index,
    anomalies['Page_Views'],
    color='red',
```

```
    label='Anomalies'
)

plt.title('Page Views with Anomalies')
plt.xlabel('Date')
plt.ylabel('Page Views')
plt.legend()
plt.show()

# Display anomalies
print("Anomalies:")
print(anomalies[['Page_Views', 'Page_Views_Z']])
```

Output :



Result :

Thus, Time Series Analysis of Website Traffic were Programmed.

Experiment 7:. Random Sampling and Sampling Distribution

Aim :

To explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

Program Code :

```
import numpy as np

import matplotlib.pyplot as plt



# Step 1: Generate a population

population_mean = 70

population_std = 15

population_size = 50000


population = np.random.normal(

    population_mean,

    population_std,

    population_size

)


# Step 2: Random sampling

sample_sizes = [20, 40, 80]

num_samples = 500


sample_means = {}


for size in sample_sizes:

    sample_means[size] = []
```

```
    for _ in range(num_samples):
        sample = np.random.choice(
            population,
            size=size,
            replace=False
        )

        sample_means[size].append(np.mean(sample))

# Step 3: Plot sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)

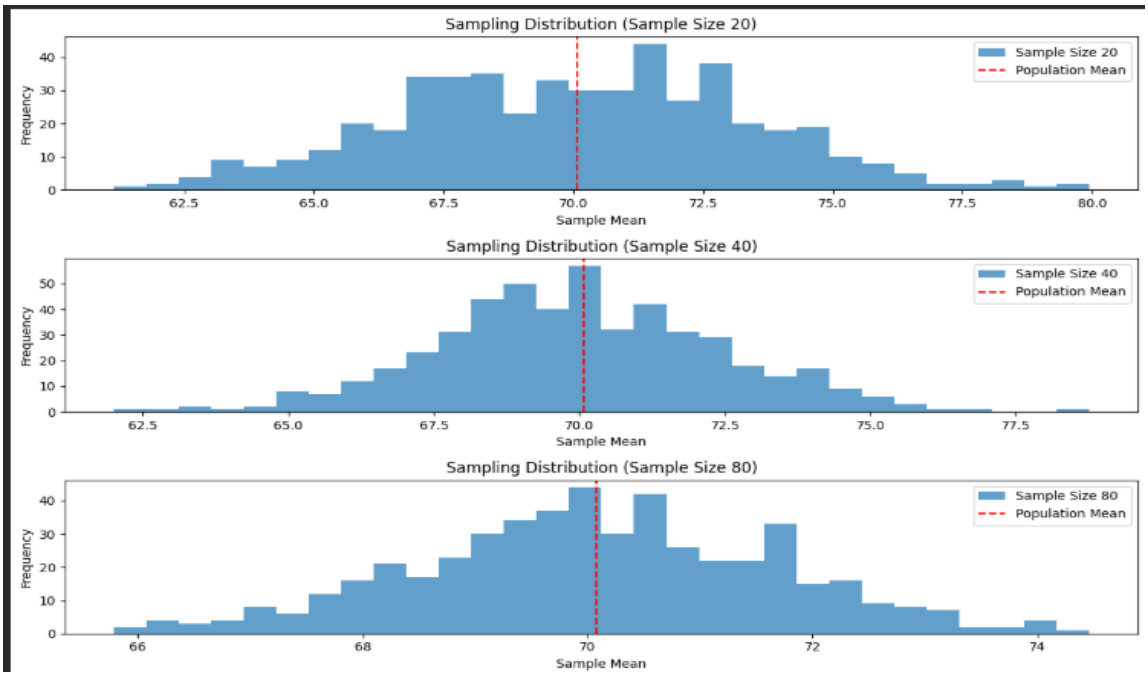
    plt.hist(
        sample_means[size],
        bins=30,
        alpha=0.7,
        label=f"Sample Size {size}"
    )

    plt.axvline(
        np.mean(population),
        color="red",
        linestyle="dashed",
        linewidth=1.5,
        label="Population Mean"
    )
```

```
    plt.title(f"Sampling Distribution (Sample Size {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.legend()

plt.tight_layout()
plt.show()
```

Output :



Result :

Thus, explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

Experiment 8: Conduct Z test for the Data Given using Python Objective:

Aim :

To test whether the average weight of a species of birds differs from 150 grams.

Program Code :

```
import numpy as np
import scipy.stats as stats

# Define the sample data (weights in grams)
sample_data = np.array([
    152, 148, 151, 149, 147, 153, 150, 148, 152, 149,
    151, 150, 149, 152, 151, 148, 150, 152, 149, 150,
    148, 153, 151, 150, 149, 152, 148, 151, 150, 153
])

# Population mean under the null hypothesis
population_mean = 150

# Calculate sample statistics
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)

# Number of observations
n = len(sample_data)

# Calculate the Z-statistic
z_statistic = (sample_mean - population_mean) / (
    sample_std / np.sqrt(n)
)
```

```
# Calculate the p-value
p_value = 2 * (1 - stats.norm.cdf(abs(z_statistic)))

# Print results
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

# Significance level
alpha = 0.05

# Decision
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is significantly different from 150 grams.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.")
```

Output :

Sample Mean: 150.20

Z-Statistic: 0.6406

P-Value: 0.5218

Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.

Experiment 9 : Experiment to understand Matplotlib library use cases in Data Science

through visualization

Aim :

Experiment to understand Matplotlib library use cases in Data Science

through visualization

Description: Use Iris data set to understand the Matplot lib library

Program Code :

```
from google.colab import files

uploaded = files.upload()
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

# Read dataset
data = pd.read_csv('/content/Iris_Dataset.csv')

data

# Dataset information
data.info()

# Statistical description
data.describe()

# Count varieties
```

```
data.value_counts('variety')

# Count plot
sns.countplot(x='variety', data=data)
plt.show()

# Convert categorical variable into dummy variables
dummies = pd.get_dummies(data.variety)

FinalDataset = pd.concat(
    [pd.get_dummies(data.variety),
     data.iloc[:, [0, 1, 2, 3]]],
    axis=1
)

FinalDataset.head()

# Scatter plot - Sepal Length vs Sepal Width
sns.scatterplot(
    x='SepalLengthCm',
    y='SepalWidthCm',
    hue='variety',
    data=data
)

plt.xlabel('SepalLengthCm')
plt.ylabel('SepalWidthCm')
plt.show()
```

```
# Scatter plot - Petal Length vs Petal Width
sns.scatterplot(
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='variety',
    data=data
)

plt.xlabel('PetalLengthCm')
plt.ylabel('PetalWidthCm')
plt.show()

# Pair plot
sns.pairplot(data, hue='variety', height=3)
plt.show()

# Histogram - Petal Length
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(
    sns.histplot,
    'PetalLengthCm'
).add_legend()

plt.show()

# Histogram - Petal Width
```

```
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(
    sns.histplot,
    'PetalWidthCm'
).add_legend()

plt.show()

# Histogram - Sepal Length
sns.FacetGrid(
    data,
    hue='variety',
    height=5+++
).map(
    sns.histplot,
    'SepalLengthCm'
).add_legend()

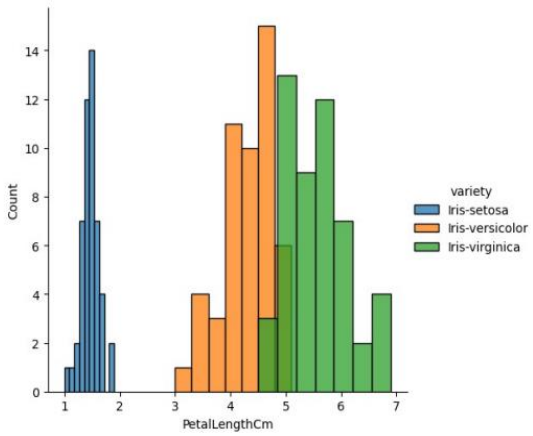
plt.show()

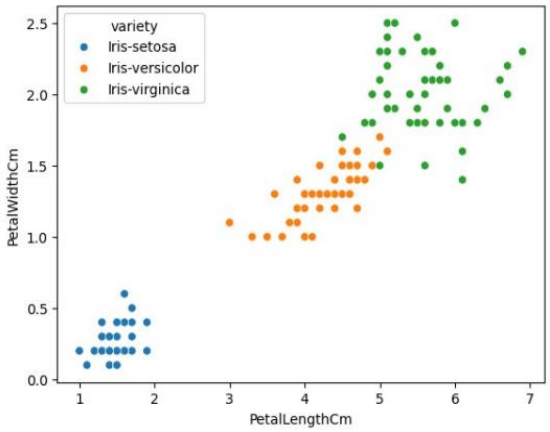
# Histogram - Sepal Width
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(
```

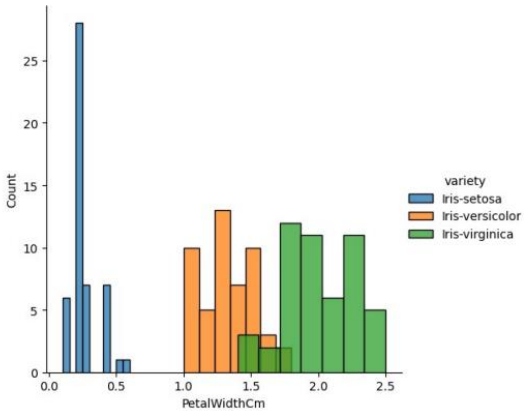
```
    sns.histplot,
    'SepalWidthCm'
).add_legend()

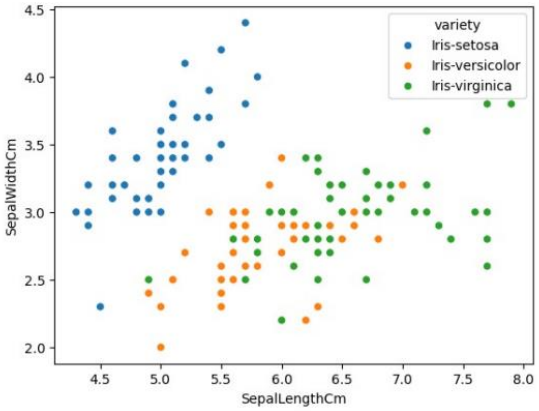
plt.show()
```

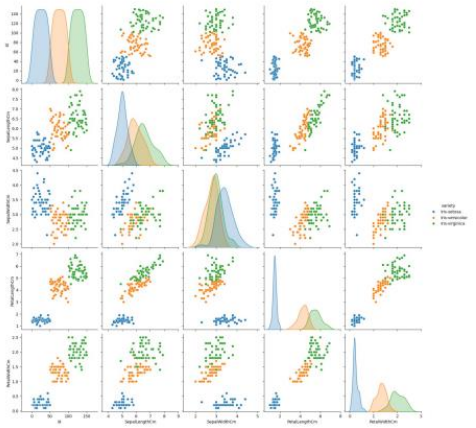
Output :

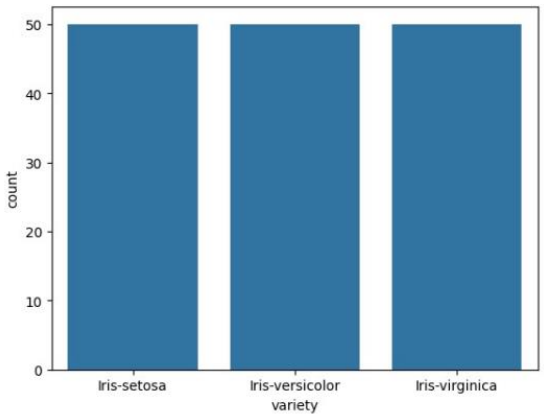


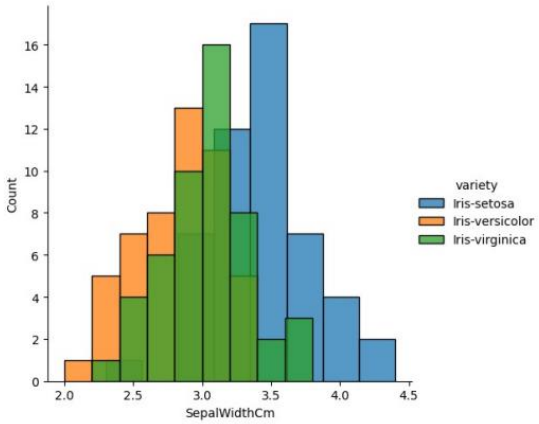


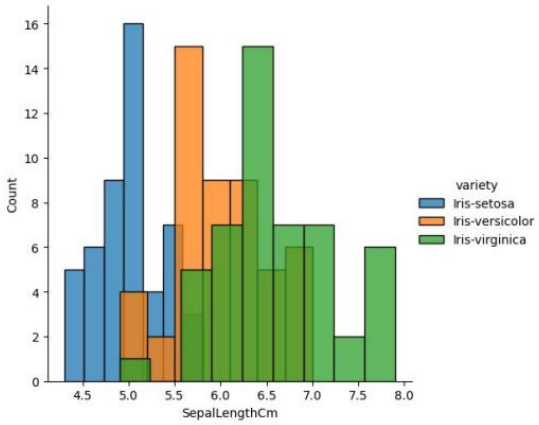












Result :

Thus,we Experiment to understand Matplotlib library use cases in Data Science through visualization

Experiment 10 : Experiment to understand array function in Data science.

Aim :

Experiment to understand array function in Data science.

Description: Understand array function using Numpy library

Program Code :

```
import numpy as np

array = np.random.randint(1, 100, 9)
print("Original Array:")
print(array)

print("\nSquare Root of Array:")
print(np.sqrt(array))

print("\nNumber of Dimensions:")
print(array.ndim)

new_array = array.reshape(3, 3)
print("\nReshaped Array:")
print(new_array)

print("\nNumber of Dimensions after Reshape:")
print(new_array.ndim)

print("\nRavel Array:")
print(new_array.ravel())

newm = new_array.reshape(3, 3)
print("\nNew Matrix:")
```

```
print(newm)

print("\nnewm[2,1:3]:")
print(newm[2, 1:3])

print("\nnewm[1:2,1:3]:")
print(newm[1:2, 1:3])

print("\nnew_array[0:3,0:0]:")
print(new_array[0:3, 0:0])

print("\nnew_array[0:2,0:1]:")
print(new_array[0:2, 0:1])

print("\nnew_array[0:3,0:1]:")
print(new_array[0:3, 0:1])

print("\nnew_array[1:3]:")
print(new_array[1:3])
```

Output :

```
Original Array:
[97 29  7 27 94  2 28 25 35]

Square Root of Array:
[9.8488578  5.38516481 2.64575131 5.19615242 9.69535971 1.41421356
 5.29150262 5.         5.91607978]

Number of Dimensions:
1
```

Reshaped Array:

[[97 29 7]

 [27 94 2]

 [28 25 35]]

Number of Dimensions after Reshape:

2

Ravel Array:

[97 29 7 27 94 2 28 25 35]

New Matrix:

[[97 29 7]

 [27 94 2]

 [28 25 35]]

newm[2,1:3]:

[25 35]

newm[1:2,1:3]:

[[94 2]]

new_array[0:3,0:0]:

[]

new_array[0:2,0:1]:

[[97]

 [27]]

new_array[0:3,0:1]:

[[97]

 [27]

 [28]]

new_array[1:3]:

[[27 94 2]

 [28 25 35]]

Result :

Thus,the we understand array function in Data science.

EXP NO: 11 Experiment to understand pandas library use cases in Data science.

AIM:

To study and experiment with the Pandas library in Python to understand its various use cases in Data Science, including data manipulation, data cleaning, data analysis, and handling structured datasets using Series and DataFrame.

PROGRAM:

```
import numpy as np

import pandas as pd

from google.colab import files


uploaded = files.upload()


list1 = [

   [1, 'Smith', 50000],

   [2, 'Jones', 60000]

]


df = pd.DataFrame(list1)


print("Initial DataFrame:")

print(df)


df.columns = ['Empd', 'Name', 'Salary']


print("\nDataFrame with Column Names:")

print(df)


print("\nDataFrame Information:")

df.info()
```

```
df = pd.read_csv("50_Startups.csv")

print("\nFirst 5 rows of 50_Startups:")
print(df.head())

print("\nLast 5 rows of 50_Startups:")
print(df.tail())

print("\nInformation of 50_Startups:")
df.info()

df = pd.read_csv("employee.csv")

print("\nFirst 5 rows of Employee:")
print(df.head())

print("\nLast 5 rows of Employee:")
print(df.tail())

print("\nEmployee Data Information:")
df.info()

print("\nSalary:")
print(df.salary)

print("\nType of Salary:")
print(type(df.salary))
```

```
print("\nMean Salary:")
print(df.salary.mean())

print("\nMedian Salary:")
print(df.salary.median())

print("\nMode Salary:")
print(df.salary.mode())

print("\nSalary Variance:")
print(df.salary.var())

print("\nSalary Standard Deviation:")
print(df.salary.std())

print("\nDescribe:")
print(df.describe())

print("\nDescribe All:")
print(df.describe(include='all'))

print("\nColumn Names:")
empCol = df.columns
print(empCol)

print("\nNumPy Array:")
emparray = df.values
```

```
print(emparray)


employee_DF = pd.DataFrame(emparray, columns=empCol)


print("\nEmployee DataFrame:")
print(employee_DF)
```

OUTPUT:

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 50_Startups.csv to 50_Startups (2).csv

Saving employee.csv to employee (1).csv

Initial DataFrame:

** 0 1 2**

0 1 Smith 50000

1 2 Jones 60000

DataFrame with Column Names:

** Empd Name Salary**

0 1 Smith 50000

1 2 Jones 60000

DataFrame Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2 entries, 0 to 1

Data columns (total 3 columns):

Column Non-Null Count Dtype

--- ------ -------------- -----

```
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
```

First 5 rows of 50_Startups:

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

Last 5 rows of 50_Startups:

	R&D Spend	Administration	Marketing Spend	State	Profit
45	1000.23	124153.04	1903.93	California	64926.08
46	1315.46	115816.21	297114.46	Florida	49490.75
47	0.00	135426.92	0.00	California	42559.73
48	542.05	51743.15	0.00	New York	35673.41
49	0.00	116983.80	0.00	California	14681.40

Information of 50_Startups:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
```

```
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB

First 5 rows of Employee:
   emp id                name  salary
0       1     SREE VARSSINI K S    5000
1       2          SREEMATHI B    6000
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000

Last 5 rows of Employee:
   emp id                name  salary
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000

Employee Data Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
```

```
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      object
 2   salary  7 non-null      int64
dtypes: int64(2), object(1)
memory usage: 300.0+ bytes

Salary:
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64

Type of Salary:
<class 'pandas.core.series.Series'>

Mean Salary:
5714.285714285715

Median Salary:
6000.0

Mode Salary:
```

0 5000

1 6000

Name: salary, dtype: int64

Salary Variance:

2571428.5714285714

Salary Standard Deviation:

1603.5674514745463

Describe:

	emp id	salary
count	7.000000	7.000000
mean	4.000000	5714.285714
std	2.160247	1603.567451
min	1.000000	3000.000000
25%	2.500000	5000.000000
50%	4.000000	6000.000000
75%	5.500000	6500.000000
max	7.000000	8000.000000

Describe All:

	emp id	name	salary
count	7.000000	7	7.000000
unique	NaN	6	NaN
top	NaN	SRI HARSHAVARDHANAN R	NaN
freq	NaN	2	NaN
mean	4.000000	NaN	5714.285714

```
std     2.160247              NaN  1603.567451
min     1.000000              NaN  3000.000000
25%     2.500000              NaN  5000.000000
50%     4.000000              NaN  6000.000000
75%     5.500000              NaN  6500.000000
max     7.000000              NaN  8000.000000

Column Names:
Index(['emp id', 'name', 'salary'], dtype='object')

NumPy Array:
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHANAN R' 3000]
 [7 'SRI HARSHAVARDHANAN R' 6000]]

Employee DataFrame:
  emp id                name salary
0      1     SREE VARSSINI K S   5000
1      2          SREEMATHI B   6000
2      3              SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5         SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

RESULT:

Thus, the experiment was successfully performed, and the various use cases of the Pandas library in Data Science, including data manipulation, data cleaning, data analysis, and handling structured data using Series and DataFrames, were studied and understood.

EXP NO: 12 Experiment to detect outliers in a given data set.

AIM:

To detect outliers in a given dataset using statistical methods such as the Interquartile Range (IQR) method and visualize the outliers using Python libraries like Pandas and Matplotlib.

PROGRAM:

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt


# Generate random data
array = np.random.randint(1, 100, 16)


print(array)


# Mean
print("Mean:", array.mean())


# Percentiles
print("25%:", np.percentile(array, 25))
print("50%:", np.percentile(array, 50))
print("75%:", np.percentile(array, 75))
print("100%:", np.percentile(array, 100))


# Outlier detection
def outDetection(array):
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
```

```
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)

print("Lower Range:", lr)
print("Upper Range:", ur)

# Plot original data
sns.displot(array)
plt.show()

# Remove outliers
new_array = array[(array > lr) & (array < ur)]

print("After removing outliers:")
print(new_array)

# Plot new data
sns.displot(new_array)
plt.show()

# Final outlier detection
lr1, ur1 = outDetection(new_array)

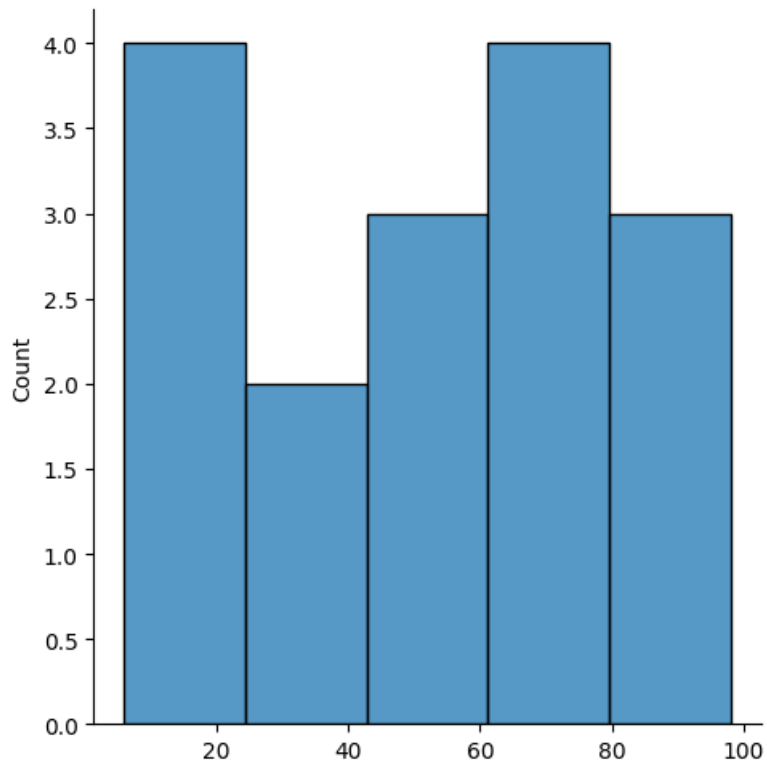
print("New Lower Range:", lr1)
print("New Upper Range:", ur1)
```

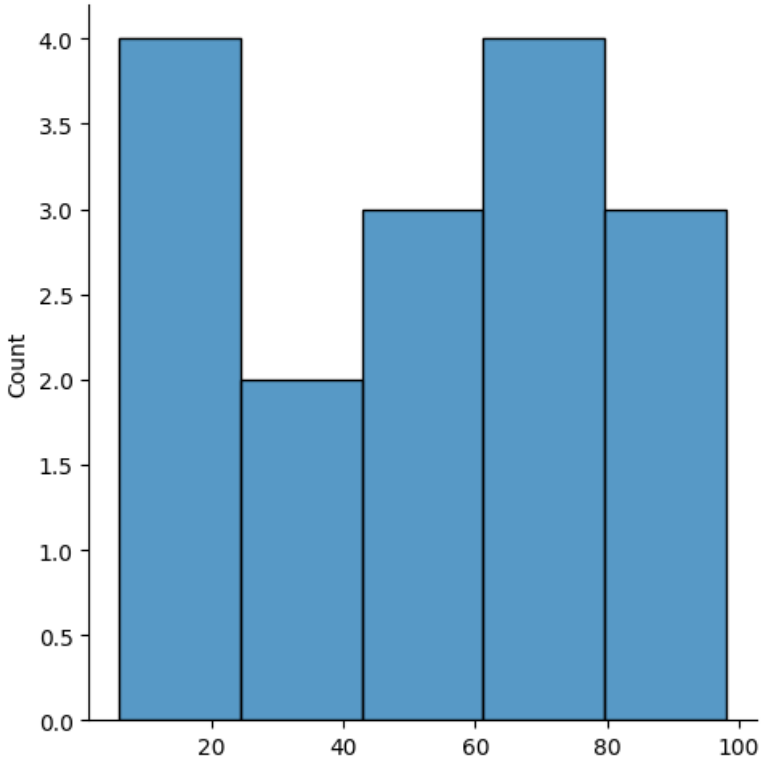
```
final_array = new_array[
    (new_array > lr1) & (new_array < ur1)
]

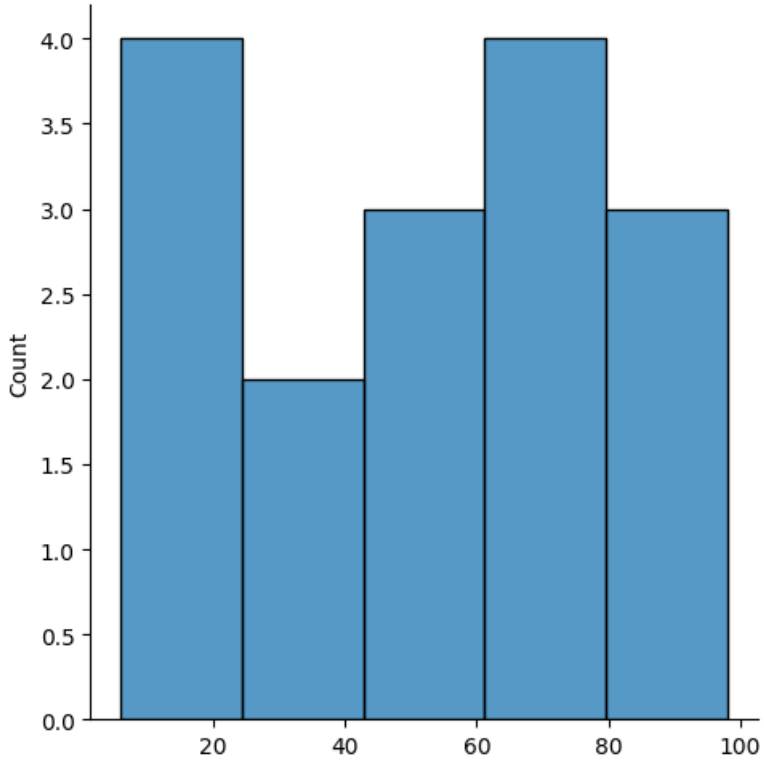
print("Final Array:")
print(final_array)

sns.displot(final_array)
plt.show()
```

OUTPUT:


Count
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
20
40
60
80
100


Count
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
20
40
60
80
100



RESULT:

Thus, the outliers in the given dataset were successfully detected using the Interquartile Range (IQR) method and visualized using Python libraries such as Pandas and Matplotlib.

EXP NO: 13 **Experiment to handle missing values and inappropriate data in a given data set**

AIM:

To identify and handle missing values and inappropriate data in a given dataset using Python libraries such as Pandas and NumPy by applying suitable data cleaning techniques, including removing, replacing, and correcting invalid values.

PROGRAM:

```
import numpy as np

import pandas as pd

from google.colab import files


uploaded = files.upload()

df = pd.read_csv("Hotel_Dataset.csv")

df


df.duplicated()


df.info()


df.drop_duplicates(inplace=True)

df


len(df)


df.reset_index(drop=True, inplace=True)

df


df.drop(["Age_Group.1"], axis=1, inplace=True)

df
```

```
df.loc[df["CustomerID"] < 0, "CustomerID"] = np.nan
df.loc[df["Bill"] < 0, "Bill"] = np.nan
df.loc[df["EstimatedSalary"] < 0, "EstimatedSalary"] = np.nan

df.loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20), "NoOfPax"] = np.nan
df

df["Age_Group"].unique()

df["Hotel"].unique()

df["Hotel"] = df["Hotel"].replace("Ibys", "Ibis")
df

df["FoodPreference"].unique()

df["FoodPreference"] = df["FoodPreference"].replace(
    ["Vegetarian", "veg"], "Veg"
)
df["FoodPreference"] = df["FoodPreference"].replace(
    "non-Veg", "Non-Veg"
)
df

df["EstimatedSalary"] = df["EstimatedSalary"].fillna(
    round(df["EstimatedSalary"].mean())
)
```

```
df["NoOfPax"] = df["NoOfPax"].fillna(
    round(df["NoOfPax"].median())
)

df["Rating(1-5)"] = df["Rating(1-5)"].fillna(
    round(df["Rating(1-5)"].median())
)

df["Bill"] = df["Bill"].fillna(
    round(df["Bill"].mean())
)

df

df.info()
print("Total rows:", len(df))
print("Total columns:", len(df.columns))
print("Missing values:")
print(df.isnull().sum())
```

OUTPUT:

```
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Age_Group.1      11 non-null     object
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             10 non-null     float64
 6   EstimatedSalary  10 non-null     float64
 7   NoOfPax          10 non-null     float64
 8   Rating(1-5)      10 non-null     float64
dtypes: float64(4), int64(1), object(4)
memory usage: 924.0+ bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       9 non-null      float64
 1   Age_Group        10 non-null     object
 2   Hotel            10 non-null     object
 3   FoodPreference   10 non-null     object
 4   Bill             10 non-null     float64
 5   EstimatedSalary  10 non-null     float64
 6   NoOfPax          10 non-null     float64
 7   Rating(1-5)      10 non-null     float64
dtypes: float64(5), object(3)
memory usage: 772.0+ bytes
Total rows: 10
Total columns: 8
Missing values:
CustomerID         1
Age_Group          0
Hotel              0
FoodPreference     0
Bill               0
EstimatedSalary    0
NoOfPax            0
Rating(1-5)        0
dtype: int64
```

RESULT:

Thus, the missing values and inappropriate data in the given dataset were successfully identified and handled using Python Pandas, and the dataset was cleaned for further analysis.

EXP NO: 14 Experiment to understand feature scaling.

AIM:

To understand and implement feature scaling techniques such as Normalization and Standardization on a given dataset using Python.

PROGRAM:

```
import numpy as np

import pandas as pd

from google.colab import files


uploaded = files.upload()

df = pd.read_csv("pre-process_datasample.csv")

df


df.head()


df.info()


df["Country"] = df["Country"].fillna(df["Country"].mode()[0])


features = df.iloc[:, :-1].values

label = df.iloc[:, -1].values


from sklearn.impute import SimpleImputer


age = SimpleImputer(strategy="mean", missing_values=np.nan)

salary = SimpleImputer(strategy="mean", missing_values=np.nan)


age.fit(features[:, [1]])
```

```
salary.fit(features[:, [2]])

features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = salary.transform(features[:, [2]])

features

from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])

Country

final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
final_set

from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)
feat_standard_scaler

from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0, 1))
```

mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)

feat_minmax_scaler

OUTPUT:

```
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

RESULT:

Thus, feature scaling techniques were successfully implemented on the given dataset to bring numerical features to a common scale.

Experiment 15 : Experiment to understand the data preprocessing in Data Science

Aim :

Experiment to understand the data preprocessing in Data Science

Description: Understand the importance of Data preprocessing in data science

Program Code:

```
import numpy as np
import pandas as pd

from google.colab import files

uploaded = files.upload()

df = pd.read_csv("/content/pre-process_datasample.csv")

print("Original Dataset:")
print(df)

print("\nDataset Information:")
df.info()

# Handle missing values
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(round(df["Salary"].mean()))
```

```
print("\nDataset After Preprocessing:")
print(df)

# One-Hot Encoding
updated_dataset = pd.concat(
    [pd.get_dummies(df["Country"]), df.iloc[:, 1:]],
    axis=1
)

# Convert Purchased into numerical values
updated_dataset["Purchased"] = updated_dataset["Purchased"].replace(
    {"No": 0, "Yes": 1}
)

print("\nFinal Dataset:")
print(updated_dataset)
```

Output :

```
Original Dataset:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
```

8 NaN 50.0 83000.0 No

9 France 37.0 67000.0 Yes

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 4 columns):

Column Non-Null Count Dtype

--- ------ -------------- -----

0 Country 9 non-null object

1 Age 9 non-null float64

2 Salary 9 non-null float64

3 Purchased 10 non-null object

dtypes: float64(2), object(2)

memory usage: 452.0+ bytes

Dataset After Preprocessing:

Country Age Salary Purchased

0 France 44.0 72000.0 No

1 Spain 27.0 48000.0 Yes

2 Germany 30.0 54000.0 No

3 Spain 38.0 61000.0 No

4 Germany 40.0 63778.0 Yes

5 France 35.0 58000.0 Yes

6 Spain 38.0 52000.0 No

7 France 48.0 79000.0 Yes

8 France 50.0 83000.0 No

9 France 37.0 67000.0 Yes

Final Dataset:

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	0
1	False	False	True	27.0	48000.0	1
2	False	True	False	30.0	54000.0	0
3	False	False	True	38.0	61000.0	0
4	False	True	False	40.0	63778.0	1
5	True	False	False	35.0	58000.0	1
6	False	False	True	38.0	52000.0	0
7	True	False	False	48.0	79000.0	1
8	True	False	False	50.0	83000.0	0
9	True	False	False	37.0	67000.0	1

Result :

Thus, Experiment to understand the data preprocessing in Data

Science

EXP NO:16 Experiment to understand EDA-Quantitative and Qualitative analysis.

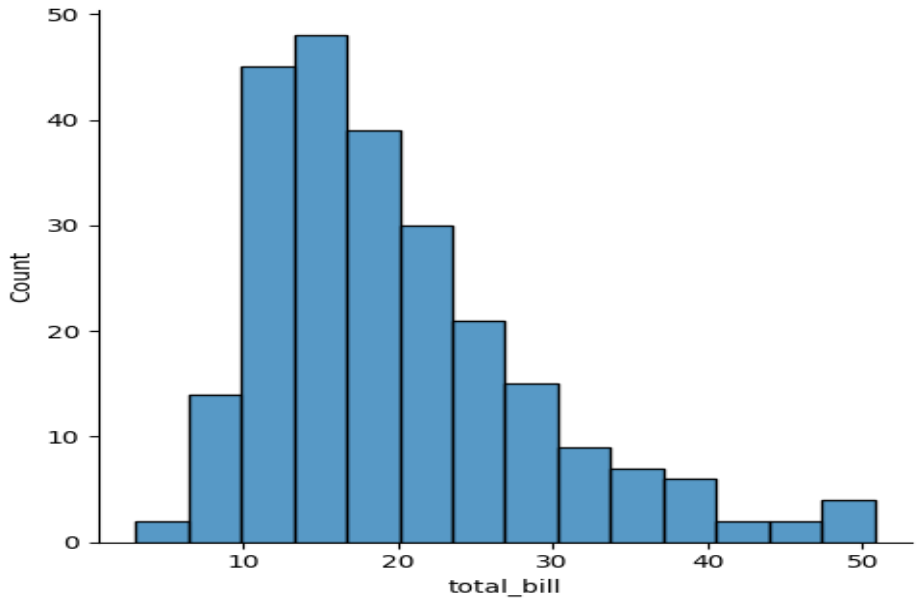
AIM:

To make an experiment to understand EDA-Quantitative and Qualitative analysis.

PROGRAM:

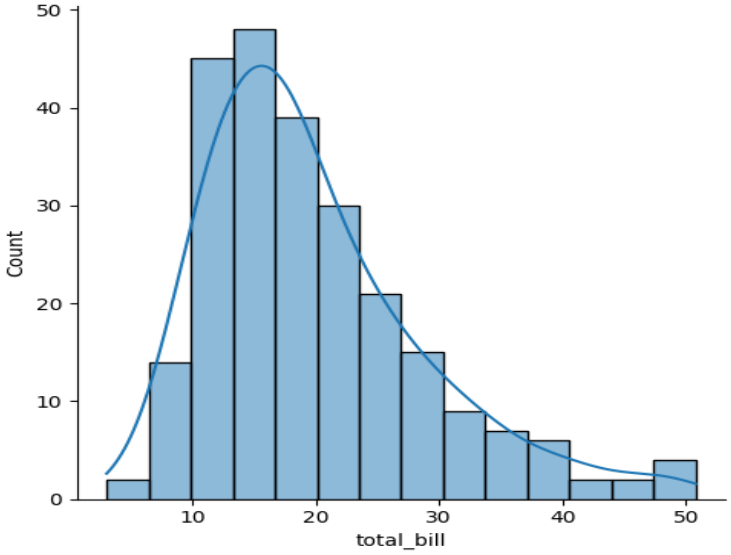
```
 import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips = sns.load_dataset('tips')
print(tips.head())
sns.displot(data=tips, x='total_bill', kde=True)
plt.show()
sns.displot(data=tips, x='total_bill', kde=False)
plt.show()
sns.jointplot(data=tips, x='tip', y='total_bill')
plt.show()
sns.jointplot(data=tips, x='tip', y='total_bill', kind='reg')
plt.show()
sns.jointplot(data=tips, x='tip', y='total_bill', kind='hex')
plt.show()
sns.pairplot(tips)
plt.show()
print(tips.time.value_counts())
sns.pairplot(tips, hue='time')
plt.show()
sns.pairplot(tips, hue='day')
plt.show()
plt.figure(figsize=(8, 6))
sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(x=tips.total_bill)
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(x=tips.tip)
plt.show()
plt.figure(figsize=(8, 5))
sns.countplot(data=tips, x='day')
plt.show()
plt.figure(figsize=(8, 5))
sns.countplot(data=tips, x='sex')
plt.show()

tips.sex.value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.show()
tips.sex.value_counts().plot(kind='bar')
```



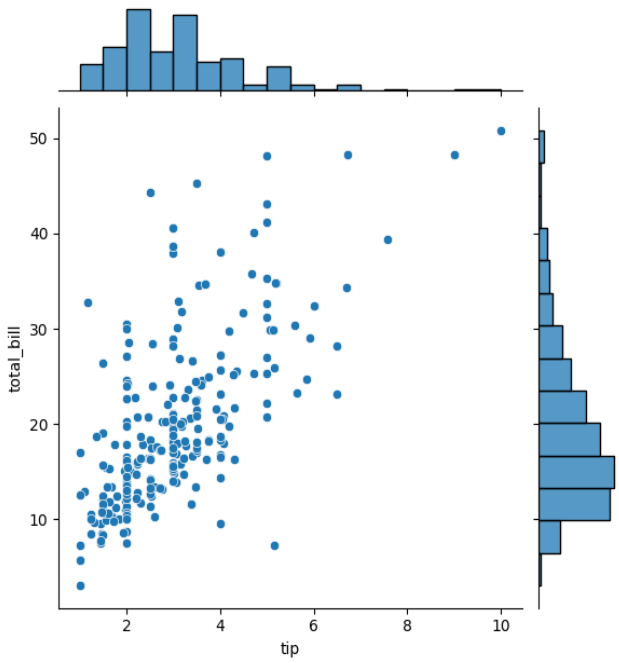
plt.show()

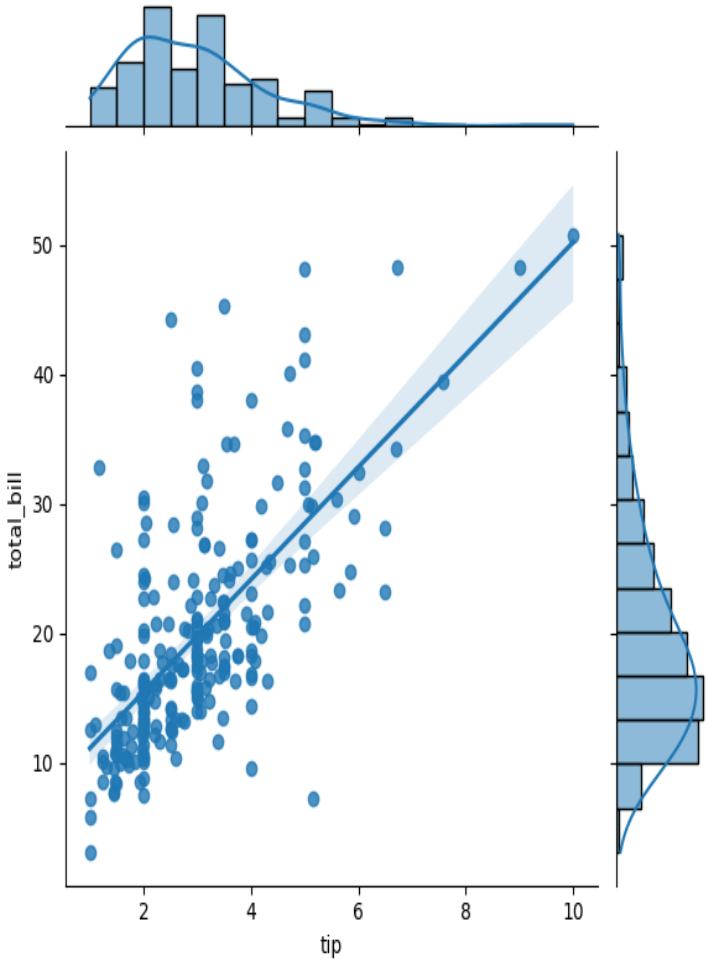
```
plt.figure(figsize=(8, 5))
sns.countplot(data=tips[tips.time == 'Dinner'], x='day')
plt.show()
```

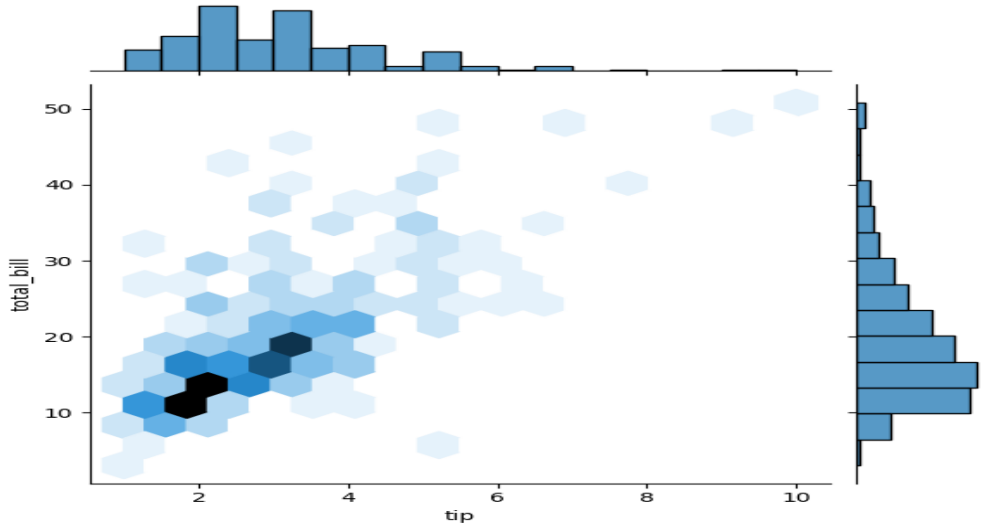


OUTPUT:

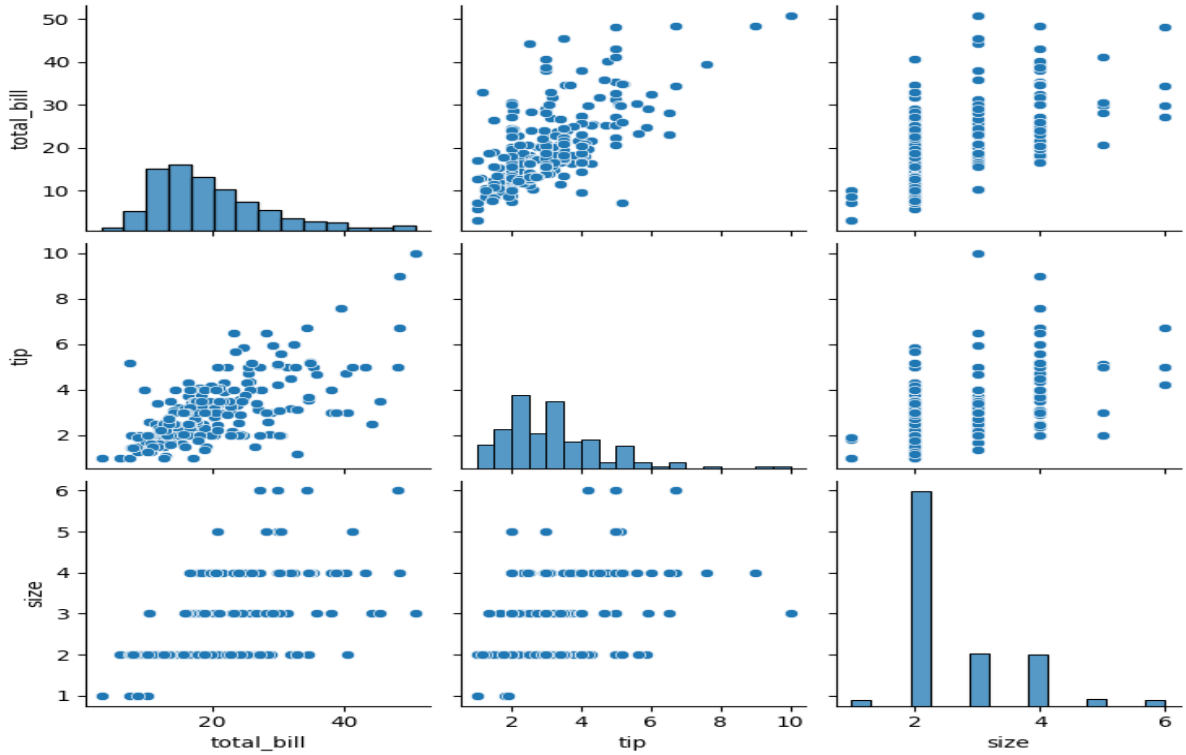
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

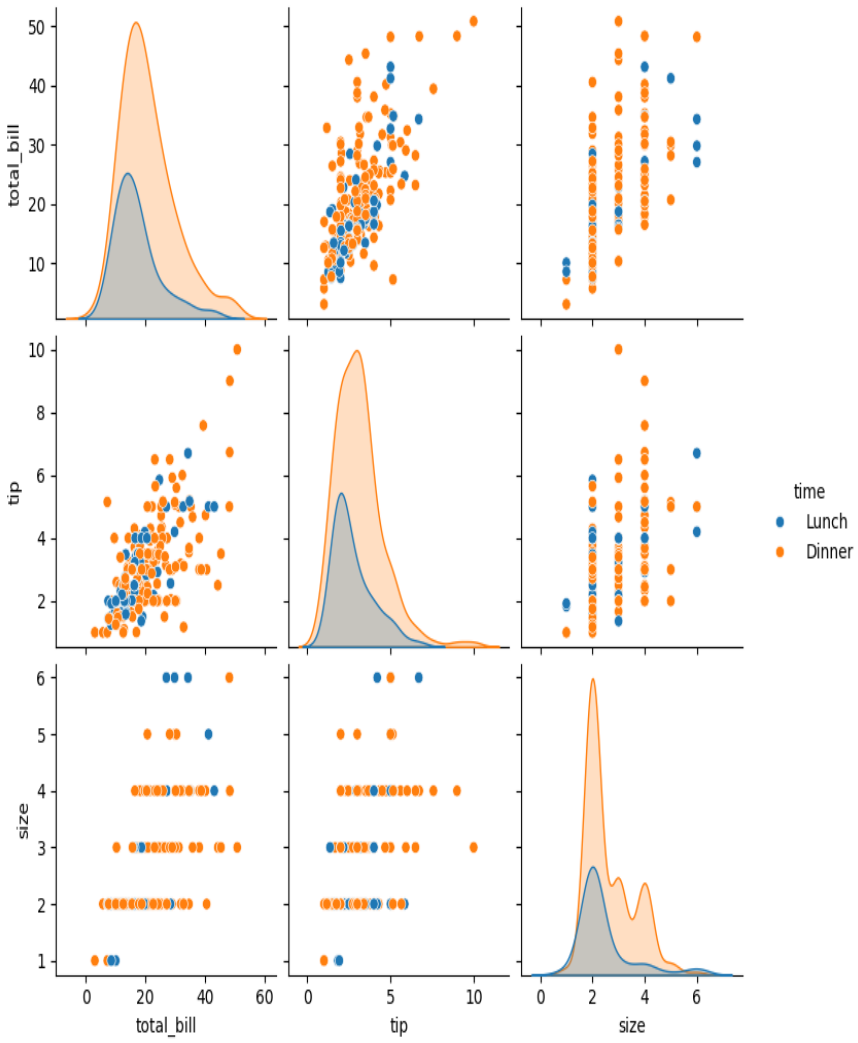

total_bill
tip
50
40
30
20
10
2
4
6
8
10

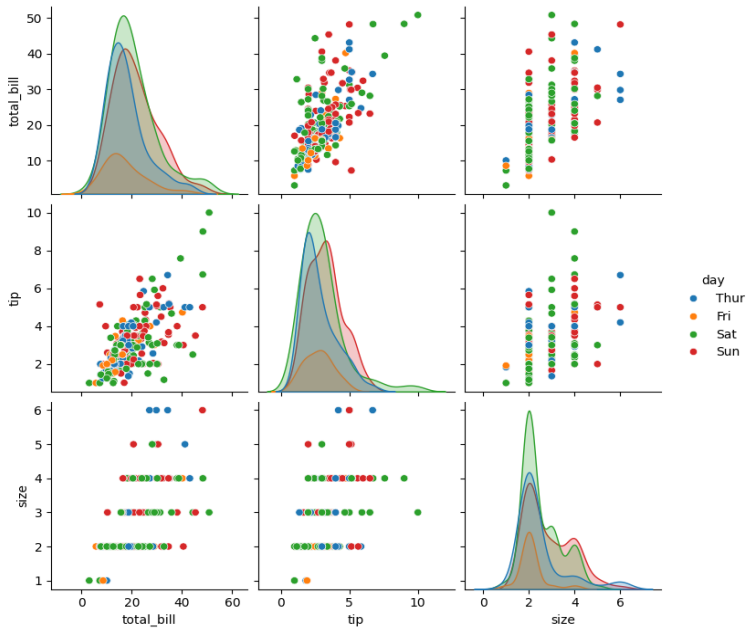

total_bill
tip
50
40
30
20
10
2
4
6
8
10

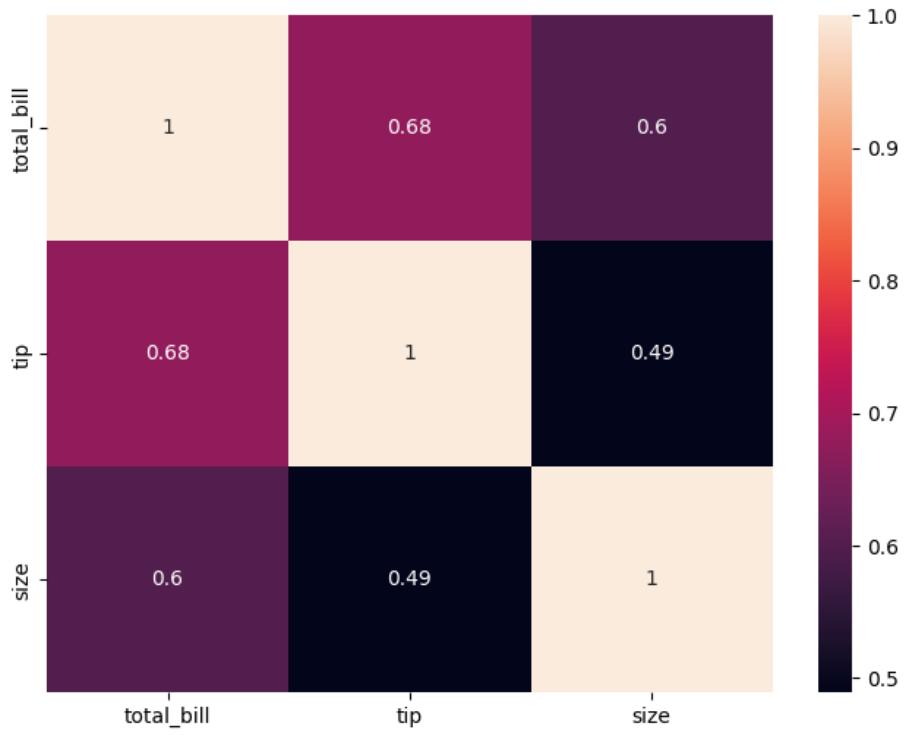


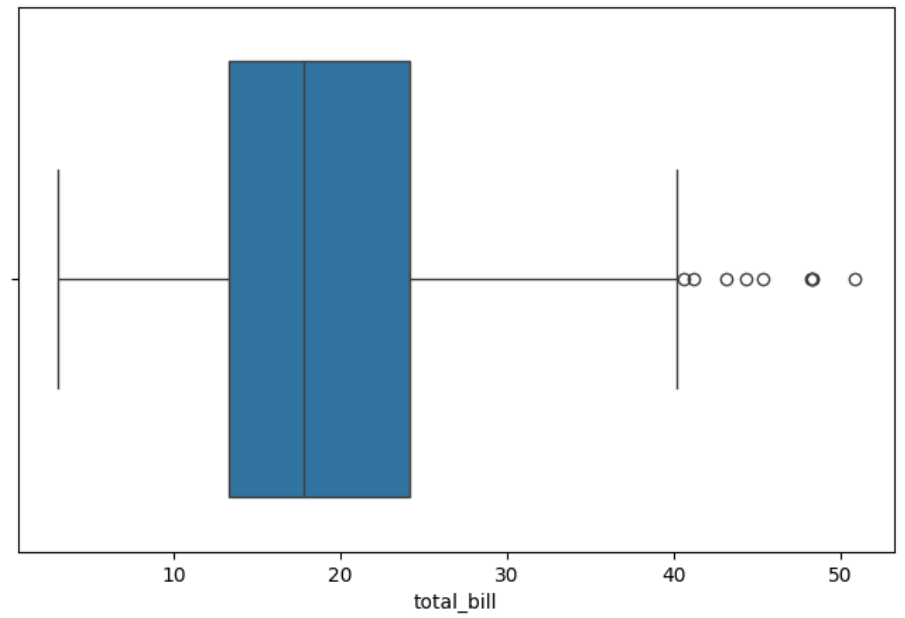
time
Dinner 176
Lunch 68
Name: count, dtype: int64

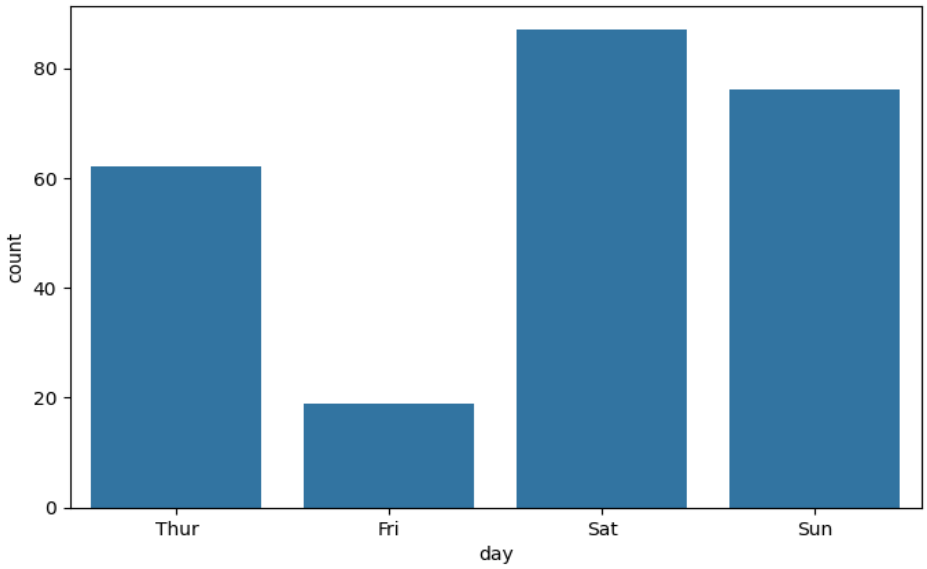


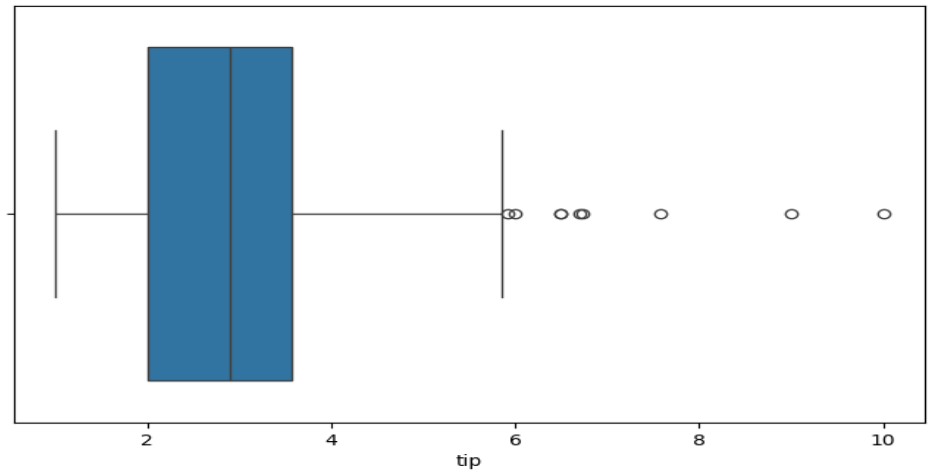

total_bill
tip
size
time
Lunch
Dinner

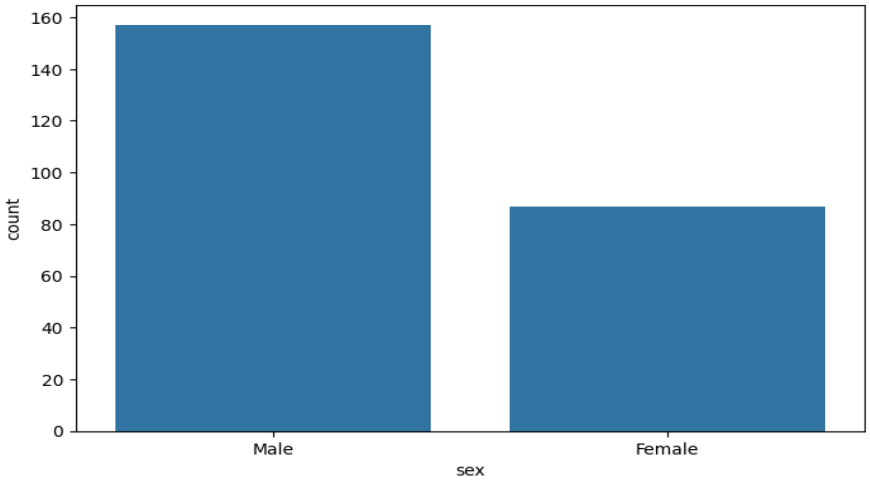

total_bill
tip
size
day
Thur
Fri
Sat
Sun

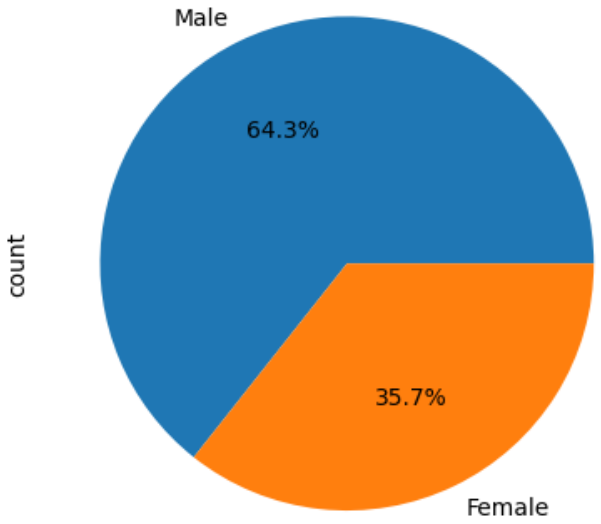

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
total_bill
tip
size
1.0
0.9
0.8
0.7
0.6
0.5

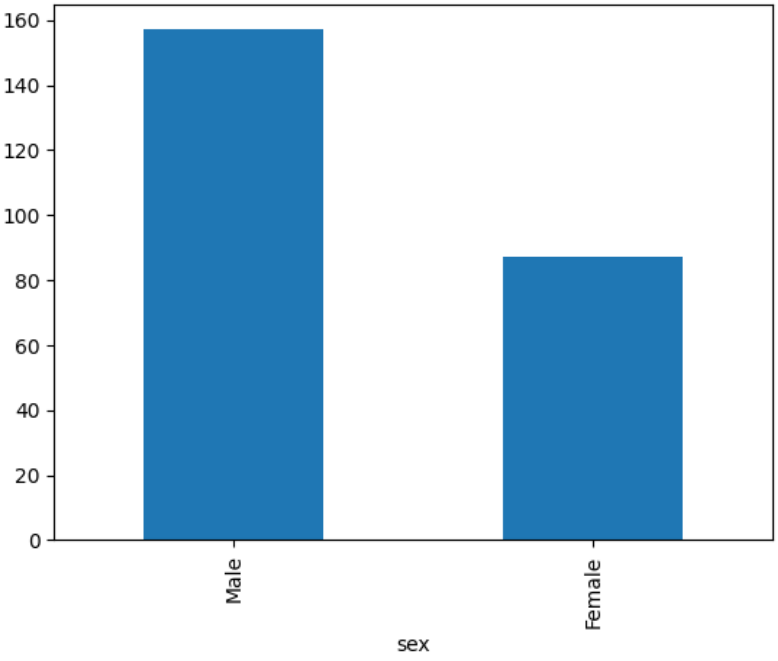

10
20
30
40
50
total_bill

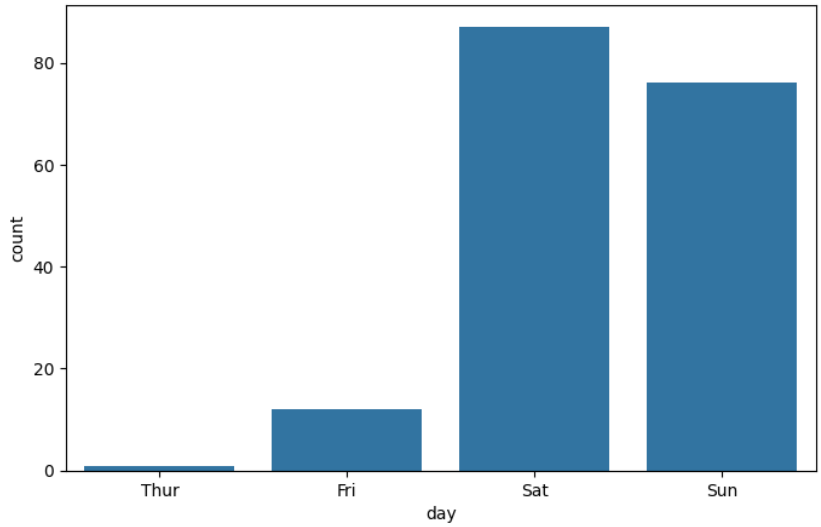

80
60
40
20
0
count
Thur
Fri
Sat
Sun
day


2
4
6
8
10
tip


160
140
120
100
80
60
40
20
0
count
Male
Female
sex


Male
64.3%
count
35.7%
Female


160
140
120
100
80
60
40
20
0
Male
Female
sex



RESULT:

Thus the experiment to understand EDA-Quantitative and Qualitative analysis is done.

EXP NO:17 Experiment to understand Linear Regression for a given data set.

AIM:

To make an experiment to understand Linear Regression for a given data set.

PROGRAM:

```
from google.colab import files
uploaded = files.upload()
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df
df.info()
df.dropna(inplace=True)
df.info()
df.describe()
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_stat
e=0)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
model.score(x_train,y_train)
model.score(x_test,y_test)
model.coef_
model.intercept_
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_np=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_np)
print("Estimated Salary for {} years of experience is {}: ".format(yr_of_exp,Salary))
```

OUTPUT:

 Salary_data.csv(text/csv) - 362 bytes, last modified: 10/12/2026 - 100% done
Saving Salary_data.csv to Salary_data.csv

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
```

```
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
Enter Years of Experience: 4.4
Estimated Salary for 4.4 years of experience is [[67755.42970824]]:
```

RESULT:

Thus the experiment to understand Linear Regression for a given data set is done.

EXP NO:18 Experiment to understand KNN algorithm for a given dataset

AIM:

To make an Experiment to understand KNN algorithm for a given dataset

PROGRAM:

```
from google.colab import files
uploaded = files.upload()
import numpy as np
import pandas as pd
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
df.columns = df.columns.str.strip()
df.info()
print(df.iloc[:, -1].value_counts())
print(df.head())
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=0
)
model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
print("Training Accuracy:", model_KNN.score(xtrain, ytrain))
print("Testing Accuracy:", model_KNN.score(xtest, ytest))
from sklearn.metrics import confusion_matrix
print("Confusion Matrix:")
print(confusion_matrix(label, model_KNN.predict(features)))
from sklzarn.metrics import classification_report

print("Classification Report:")

print(classification_report(label, model_KNN.predict(features)))
```

OUTPUT:

```
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
Training Accuracy: 0.95
Testing Accuracy: 0.9666666666666667
Confusion Matrix:
[[50  0  0]
 [ 0 45  5]
 [ 0  2 48]]
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        50
  versicolor       0.96      0.90      0.93        50
   virginica       0.91      0.96      0.93        50

    accuracy                           0.95       150
   macro avg       0.95      0.95      0.95       150
weighted avg       0.95      0.95      0.95       150
```

RESULT:

Thus the experiment to understand KNN algorithm for a given data

Set is done.

EXP NO: 19 **Experiment to understand Logistic Regression for a given data set.**

AIM:

To understand and implement Logistic Regression on a given dataset to predict categorical outcomes by establishing the relationship between input features and the target variable.

PROGRAM:

```
import numpy as np

import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import classification_report


uploaded = files.upload()

df = pd.read_csv('Social_Network_Ads.csv')


print(df.head())


features = df.iloc[:, [2, 3]].values

label = df.iloc[:, 4].values


for i in range(1, 401):

    x_train, x_test, y_train, y_test = train_test_split(

        features, label, test_size=0.2, random_state=i

    )


    model = LogisticRegression()

    model.fit(x_train, y_train)
```

```
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print("Test", test_score, "Train", train_score,
              "Random State", i)

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=37
)

finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)

print("Training Accuracy:", finalModel.score(x_train, y_train))
print("Testing Accuracy:", finalModel.score(x_test, y_test))

print(classification_report(label, finalModel.predict(features)))
```

OUTPUT:

```
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0
Training Accuracy: 1.0
Testing Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        25
           1       1.00      1.00      1.00        18

    accuracy                           1.00        43
   macro avg       1.00      1.00      1.00        43
weighted avg       1.00      1.00      1.00        43
```

RESULT:

Thus, the Logistic Regression algorithm was successfully implemented on the given dataset to predict the target variable, and the model's performance was evaluated.

EXP NO: 20 **Experiment to understand K-means clustering algorithm for a given data set. give aim for this**

AIM:

To understand and implement the K-means clustering algorithm on a given dataset to group similar data points into clusters based on their features.

PROGRAM:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.cluster import KMeans

from google.colab import files


uploaded = files.upload()

df = pd.read_csv('Mall_Customers.csv')


df.info()

print(df.head())


sns.pairplot(df)

plt.show()


features = df.iloc[:, [3, 4]].values


model = KMeans(n_clusters=5, random_state=42, n_init=10)

model.fit(features)


Final = df.iloc[:, [3, 4]].copy()
```

```
Final['label'] = model.predict(features)


print(Final.head())


sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=8) \
    .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
    .add_legend()


plt.show()


features_el = df.iloc[:, [2, 3, 4]].values
wcss = []


for i in range(1, 10):
    model = KMeans(n_clusters=i, random_state=42, n_init=10)
    model.fit(features_el)
    wcss.append(model.inertia_)


plt.plot(range(1, 10), wcss, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.show()
```

OUTPUT:

```
#   Column                Non-Null Count  Dtype
---  ------                --------------  -----
```

0 CustomerID 200 non-null int64

1 Gender 200 non-null object

2 Age 200 non-null int64

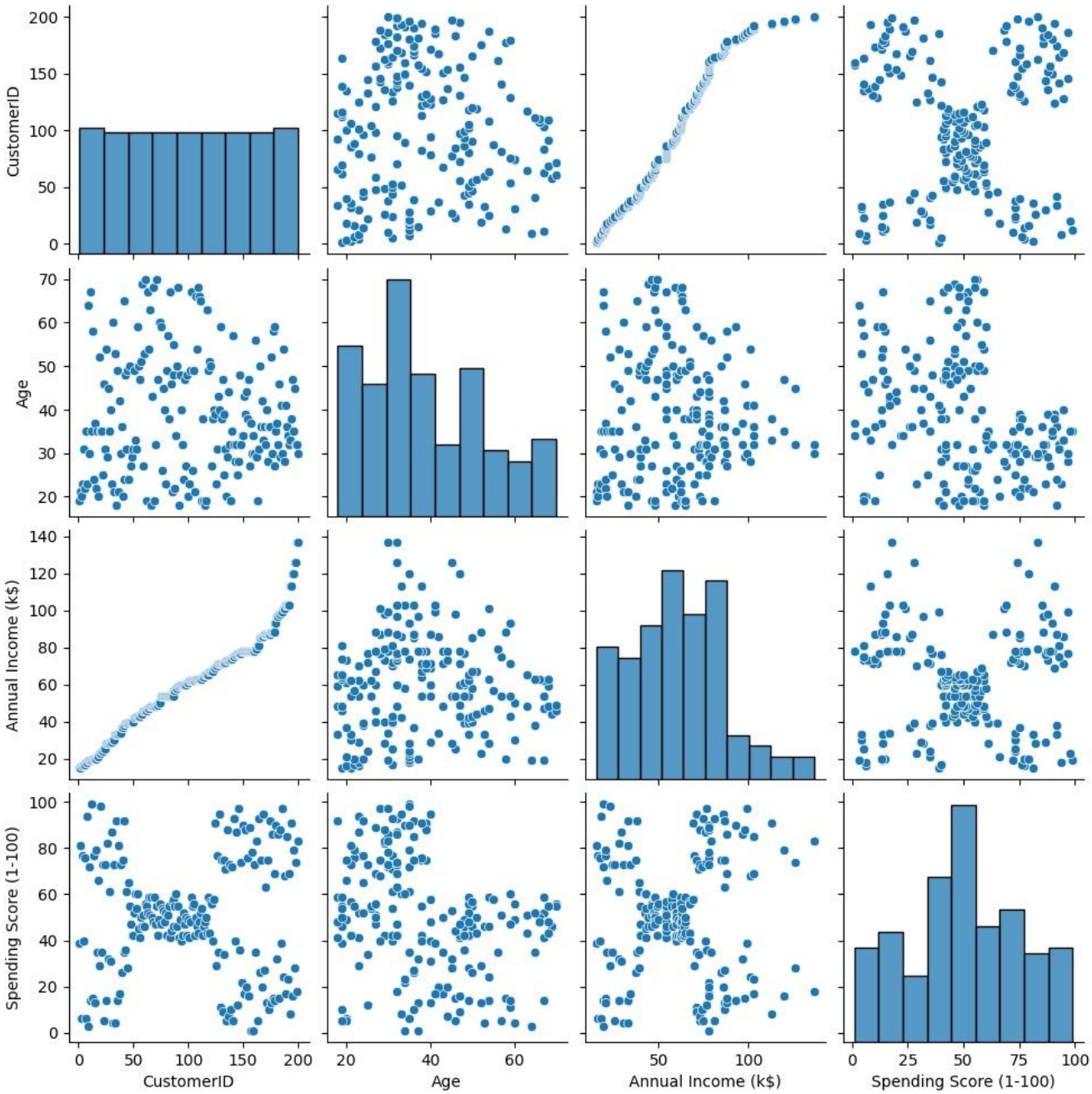
3 Annual Income (k$) 200 non-null int64

4 Spending Score (1-100) 200 non-null int64

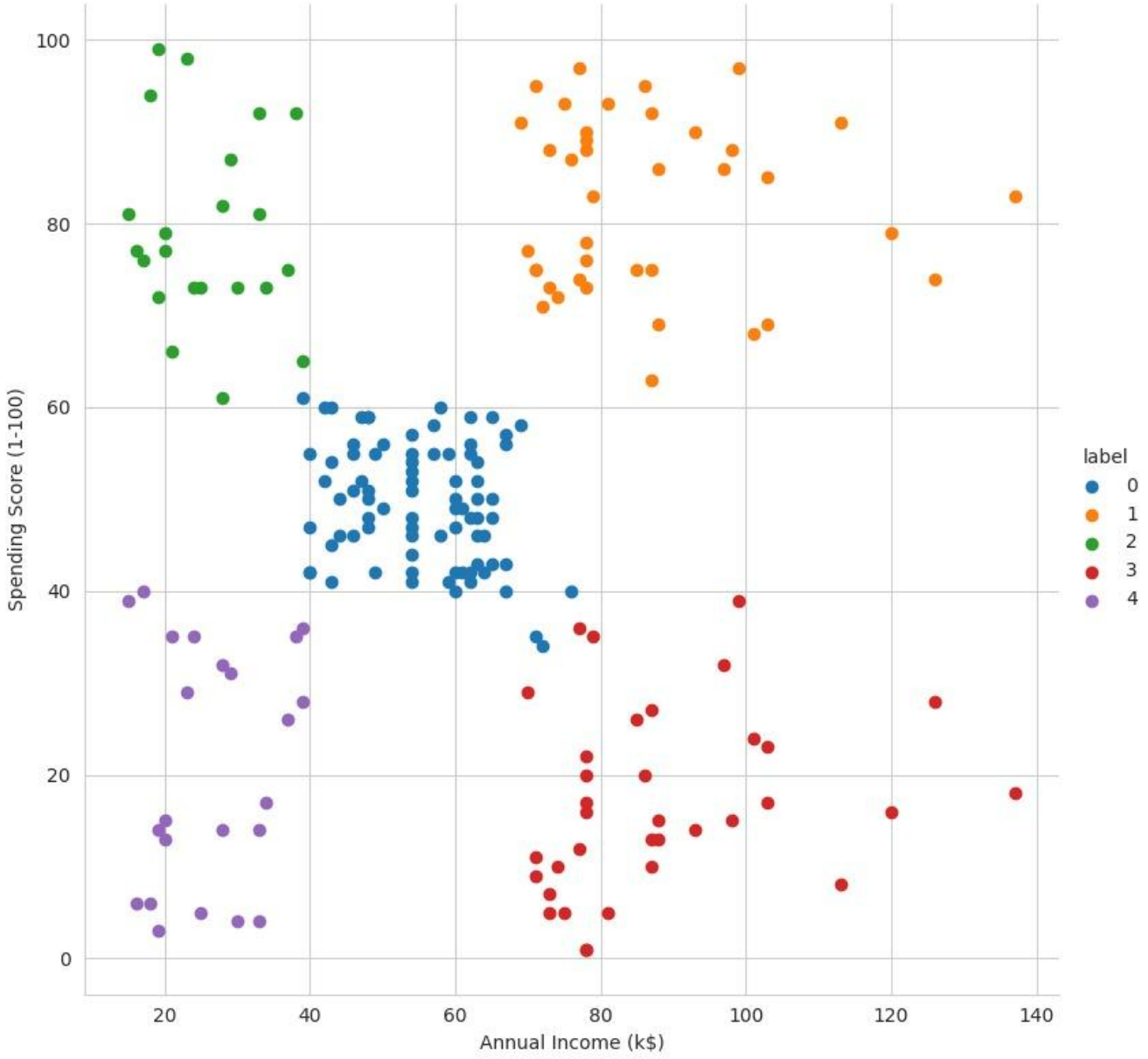
dtypes: int64(4), object(1)

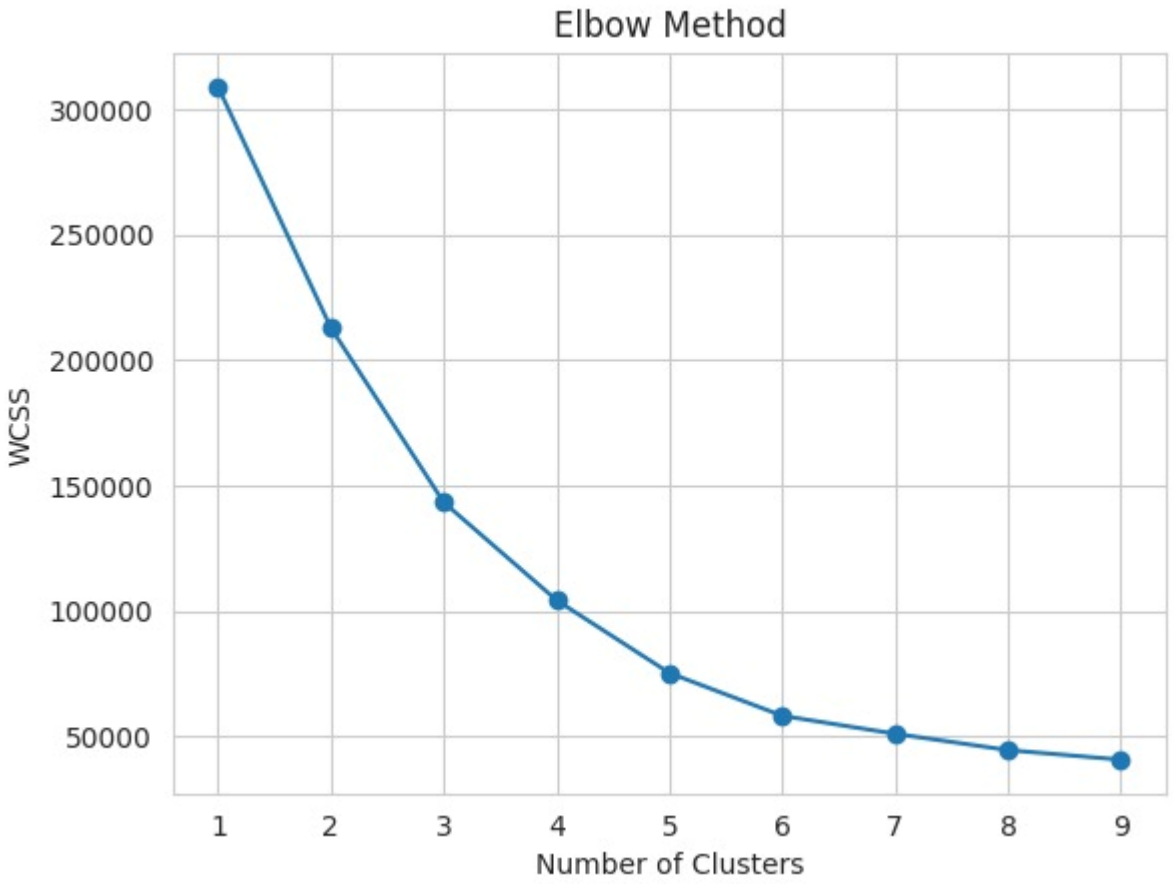
memory usage: 7.9+ KB

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40



	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4


Spending Score (1-100)
Annual Income (k$)
label
0
1
2
3
4
0
20
40
60
80
100
20
40
60
80
100
120
140



RESULT:

Thus, the K-means clustering algorithm was successfully implemented on the given dataset, and the data points were grouped into clusters based on their similarities.